Anomaly Detection in LLM-Based Systems: A Survey

CHIMING DUAN, Peking University, China
TONG JIA*, Peking University, China
MINGHUA HE, Peking University, China
LINGZHE ZHANG, Peking University, China
HAOBO WANG, Peking University, China
YING LI*, Peking University, China

Anomaly detection identifies behavior patterns that do not conform to expected behavior. It is essential for maintaining the reliability and safety of LLM-based systems. Boundary-Modeling AD follows anomaly detection approaches used in conventional software systems. It checks behavior patterns against a boundary pre-modeled from historical runtime data. However, LLM uncertainty limits how well historical normal patterns characterize expected behavior. Alignment-Based AD constructs an alignment model to evaluate current behavior patterns against applicable requirements under the execution context. Requirement violations indicate anomalies. The survey organizes existing methods into taxonomies for both paradigms and discusses challenges and future directions. Website: https://method-notes.github.io/LLM-Anomaly-Detection-Survey/

CCS Concepts: • **Software and its engineering** → **Software fault tolerance**; • **Security and privacy** → *Intrusion/anomaly detection and malware mitigation*; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: LLM-based systems, anomaly detection, AIOps, large language models, reliability, alignment-based detection

1 Introduction

Large language models (LLMs) are becoming central components of software systems. We refer to software systems that use LLMs during execution to generate outputs or determine actions as *LLM-based systems*. Prompts and AI chains become part of service logic, while large language models shape system interfaces and execution [111, 188, 200]. LLM-based systems combine retrieval, reasoning, planning, memory, tool use, and output generation in a connected execution process. The interaction among these components makes system behavior complex. An anomaly in one component can affect subsequent execution and the final system outcome. Detecting anomalies in LLM-based systems is therefore essential for meeting service-level objectives (SLOs), fulfilling user intent, and maintaining safety compliance.

Anomaly detection aims to find patterns in data that do not conform to expected behavior [17]. *Behavior patterns* characterize how a system behaves during execution. A detector models these patterns from runtime data, such as traces, logs, and metrics, and establishes *decision conditions* that characterize expected behavior. It then determines whether the current behavior pattern satisfies these conditions. The central modeling question is how these decision conditions are constructed and applied. Expected behavior can be expressed through requirements specifying the goals behavior should achieve and the constraints it should satisfy. This survey considers these goals and constraints across the SLO, user intent, and safety dimensions, covering system operation, task completion, and safe execution, respectively.

Some methods follow the anomaly detection approach used in conventional software systems. Execution in these systems largely follows predefined program logic and interfaces. For example,

*Corresponding authors: Tong Jia and Ying Li.

Authors' Contact Information: Chiming Duan, Peking University, Beijing, China, duanchiming@stu.pku.edu.cn; Tong Jia, Peking University, Beijing, China, jia.tong@pku.edu.cn; Minghua He, Peking University, Beijing, China, hemh2120@stu.pku.edu.cn; Lingzhe Zhang, Peking University, Beijing, China, zhang.lingzhe@stu.pku.edu.cn; Haobo Wang, Peking University, Beijing, China, haobo.pku@gmail.com; Ying Li, Peking University, Beijing, China, li.ying@pku.edu.cn.

a user request in a cloud-native service can trigger a predefined sequence of API calls. Under comparable operating conditions, the same request typically produces similar execution paths and invocation relations, supporting repeatable structures and sufficiently stable statistical features in runtime data. Normal behavior patterns in historical runtime data are regarded as conforming to expected behavior. Methods model the statistical, structural, or feature regularities of these data to pre-model a boundary characterizing expected behavior. The boundary specifies an accepted region, and detection checks whether subsequent behavior patterns lie within it. We characterize these methods as *Boundary-Modeling AD*.

In LLM-based systems, a normal behavior pattern observed in historical runtime data may not conform to expected behavior in the current execution. LLMs generate responses, organize reasoning, and select tools using the current request and execution context. The same user intent can be fulfilled through different retrieved evidence, reasoning paths, and tool calls. Conversely, a response format or tool-call structure resembling a historical normal execution can fail to fulfill the current request. Normal and anomalous behavior patterns can therefore overlap in statistical, structural, or feature representations, yielding a *chaotic boundary* when expected behavior is characterized from historical runtime data alone. This chaotic boundary limits the generalization of Boundary-Modeling AD detectors and can produce both false positives and false negatives. A valid reasoning path or tool-call combination can fulfill the same request as historical normal executions while lying outside the accepted statistical or feature region, producing a false positive. An execution can also reproduce historical normal response formats or tool-call structures while generating content or selecting operations that fail to fulfill the current request. Its statistical or feature representation can remain inside the accepted region, producing a false negative. This chaotic boundary differs from boundary drift caused by system updates or configuration changes. With boundary drift, new runtime data can capture the changed normal regularities and support a transition from one relatively stable boundary to another. With a chaotic boundary, the difficulty lies in establishing a stable boundary characterizing expected behavior from runtime data alone. For example, a fluent response from a retrieval-augmented generation (RAG) system must still be checked against the evidence relevant to the current request. ContextCheck combines context-grounding checks with response self-consistency checks to detect unsupported answers [122]. Similarly, a syntactically valid tool call must be related to the current task, plan, and tool specifications. These examples show why similarity to historical normal patterns alone does not establish conformity to expected behavior.

Therefore, other methods express expected behavior through requirements and align them with current behavior patterns under the execution context. We characterize these methods as *Alignment-Based AD*. Requirements specify the goals behavior should achieve and the constraints it should satisfy. The SLO, user intent, and safety dimensions describe these goals and constraints for system operation, task completion, and safe execution, respectively. Requirements expressed in natural language, rules, or logical conditions and behavior patterns modeled from runtime data can have different representations. An *alignment model* establishes their correspondences and decision conditions under the current execution context. The correspondences connect the requirements to relevant entities, states, actions, or relations in the behavior pattern. The decision conditions specify what the pattern must satisfy. Detection evaluates these conditions, and the model is updated as requirements, runtime data, and execution context change. Figure 1 illustrates these two ways of constructing and applying decision conditions.

Existing studies often focus on particular anomalies, including hallucinations, grounding failures, and unsafe tool use [83, 118, 122]. These studies are commonly described by anomaly type or detection technique. This survey connects them through how their detectors establish decision conditions characterizing expected behavior. We organize methods into Boundary-Modeling AD

and Alignment-Based AD and compare how their models are constructed, evaluated, and updated. Figure 2(a) summarizes the 181 taxonomy examples in the current main article, with 108 assigned to Boundary-Modeling AD and 73 to Alignment-Based AD. The quarterly counts in Figure 2(b) cover the 173 examples with recorded publication or preprint dates through 2026 Q2. These counts describe the selected examples rather than the overall publication population.

- **Scope.** This survey focuses on anomaly detection *of* LLM-based systems as defined in Section 2.1. The scope covers LLM-based applications, RAG systems, and LLM-based agent systems. The scope also includes runtime components for LLM serving, orchestration, observability, verification, and governance. Generated outputs, reasoning traces, agent traces, and system metrics are treated as observable data produced by these systems. Included studies must satisfy the definition of anomaly detection by identifying behavior patterns in runtime data from LLM-based systems that do not conform to expected behavior. We do not require these studies to use the term *anomaly detection*. Expected behavior is expressed through requirements in the three dimensions of SLO, user intent, and safety. This criterion includes studies described as hallucination detection, grounding verification, runtime monitoring, failure diagnosis, guardrails, or policy verification when they perform this anomaly-detection function.
- **Survey Methods.** We searched ACM Digital Library, IEEE Xplore, Scopus, Web of Science, and arXiv for records from January 2022 available at the July 10, 2026 retrieval cutoff. This period covers the current generation of LLM-based systems. Earlier foundational studies were identified separately through backward reference tracing. The search query combined an LLM-based system block with a detection block. As shown in Figure 3, LLM-based system terms, including LLM, RAG, AI agent, tool use, and LLMOps, were searched in all fields to improve recall. Detection- and harness-related terms were restricted to the

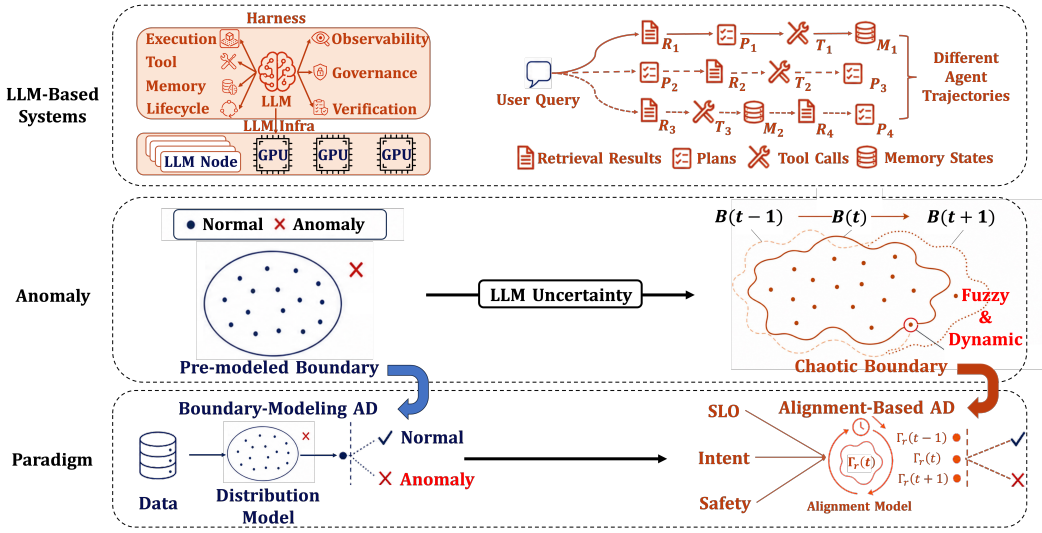
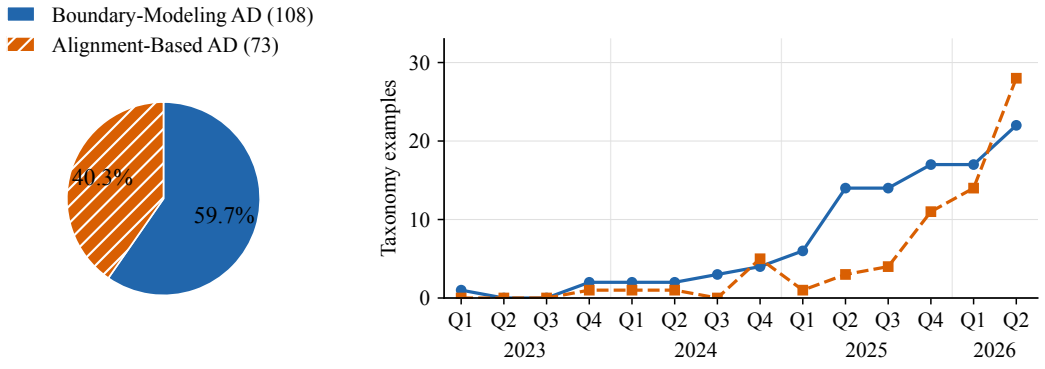


Fig. 1. Motivation for two anomaly-detection paradigms in LLM-based systems. Normal and anomalous behavior patterns can overlap when expected behavior is characterized from historical runtime data alone, giving rise to a chaotic boundary. Boundary-Modeling AD checks patterns against a pre-modeled boundary. Alignment-Based AD constructs and updates an alignment model linking requirements with current behavior patterns to establish decision conditions.



(a) Paradigm composition **(b) Quarterly counts**

Fig. 2. Composition and temporal distribution of the survey's taxonomy examples. Panel (a) groups all 181 examples by principal paradigm; panel (b) shows quarterly counts for the 173 examples with recorded publication or preprint dates through 2026 Q2. These counts describe the selected examples, not the overall publication population.

title field to improve relevance. The search identified 5,658 records across the five sources. After deduplication, 4,157 records remained. We screened their titles and abstracts against the survey scope and retained 1,295 records for further assessment. Further classification and abstract screening yielded 662 method-candidate records. We selected representative examples based on methodological distinctiveness and taxonomy coverage. The current main article contains 181 taxonomy examples; supporting methods, evaluation resources, and adjacent work are recorded separately. Studies that repeated an existing mechanism in another model, task, or domain were not discussed individually.

— **Contribution.** We organize anomaly detection methods for LLM-based systems into two paradigms based on how they establish decision conditions. *Boundary-Modeling AD* constructs a boundary from the statistical, structural, or feature regularities of historical runtime data. *Alignment-Based AD* constructs and updates an alignment model connecting requirements with current behavior patterns under the execution context. We develop taxonomies for both paradigms and analyze the observable data, anomaly locations, and anomaly types covered by existing methods. A five-stage framework describes requirement instantiation, behavior representation, model construction, evaluation, and evolution. We further discuss challenges in constructing and evaluating these models across interacting requirements and changing execution conditions.

— **Organization.** Section 2 introduces anomaly detection and LLM-based systems and defines the concepts used throughout this survey. Section 3 reviews Boundary-Modeling AD, and Section 4 reviews Alignment-Based AD. Section 5 discusses challenges and future research directions. Section 6 concludes the survey.

2 Background

This section introduces LLM-based systems and the two anomaly detection paradigms. It then describes observable data, datasets, and measurements and discusses related research domains.

ALL_FIELD("Large Language Model" **OR** "LLM" **OR** "Foundation Model" **OR** "AI Agent" **OR** "Agentic AI" **OR** "Retrieval-Augmented Generation" **OR** "RAG" **OR** "Tool Use" **OR** "LLMOps") **AND TITLE**("Anomaly Detection" **OR** "Failure Detection" **OR** "Runtime Monitoring" **OR** "Failure Diagnosis" **OR** "Hallucination Detection" **OR** "Guardrail" **OR** "Verification" **OR** "Harness")

Fig. 3. Normalized search strategy for anomaly detection in LLM-based systems.

2.1 LLM-Based Systems

An *LLM-based system* integrates one or more LLMs as runtime components whose outputs contribute to responses or guide subsequent execution. Examples include LLM applications, RAG systems, and LLM-based agent systems. Prompts and AI chains can form part of service logic [188]. Foundation models shape system boundaries, interfaces, and capabilities [111, 200]. Cloud-native mechanisms can provide the runtime infrastructure [112]. A typical LLM-based agent system can organize its runtime components into a seven-layer harness covering execution environment, tool interface, context and memory, lifecycle and orchestration, observability, verification, and governance [96]. The harness connects model outputs to actions, maintains execution state, and checks applicable requirements. Instructions, retrieved evidence, memory, user state, and policies guide response generation, planning, and tool use. Execution and verification return observations for subsequent steps. The observability layer records model calls, retrieval steps, tool invocations, and context assembly as structured spans with latency, token usage, cost, errors, and retries. Agent-level records capture identities, roles, workflows, and state handoffs.

Let observable data at time t be

$$z_t = (\mathcal{S}_t, \mathcal{A}_t), \qquad z_{0:T} = (z_0, z_1, \ldots, z_T), \tag{1}$$

where $\mathcal{S}_t$ contains structured spans and $\mathcal{A}_t$ their attributes. These data cover reasoning artifacts, tool and API interactions, and contextual information [96]. Anomaly detection uses these observable data to identify behavior patterns that do not satisfy the applicable SLO, user intent, or safety requirements. Its results can support failure diagnosis and guide intervention during execution.

2.2 Anomaly Detection Paradigms

Anomaly detection identifies patterns that do not conform to expected behavior [17]. In LLM-based systems, it supports SLO, user intent, and safety by detecting violations of the corresponding requirements. SLO requirements define service targets such as latency, availability, and cost. Detection identifies behavior that violates these targets [69, 87]. User intent specifies the requested outcome and constraints on achieving it. Detection identifies failures to fulfill the request or violations of the user's constraints [171]. Safety requirements govern permissions, security, privacy, and execution policies. Detection identifies unauthorized or unsafe behavior that violates these requirements [18]. These *end-to-end requirements* express goals and constraints for system execution. A *requirement instance* specifies those applicable to the current request and execution context. Boundary-Modeling AD and Alignment-Based AD differ in how they establish decision conditions for evaluating behavior patterns.

2.2.1 Boundary-Modeling AD. Boundary-Modeling AD follows prior work on anomaly detection in software systems such as cloud-native services. It constructs a boundary characterizing expected behavior by modeling statistical distributions, structural regularities, or anomaly-related features from historical runtime data. Predefined rules can guide feature construction. Thresholds or other

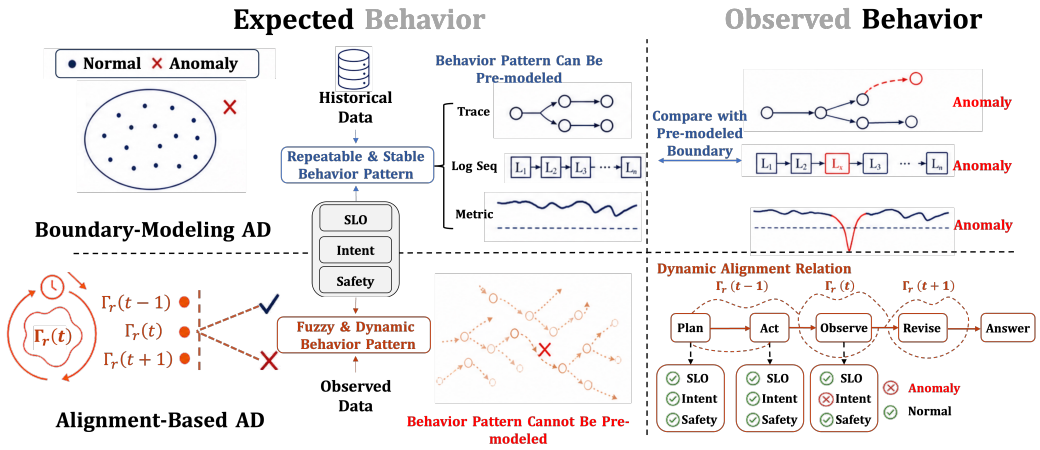


Fig. 4. Comparison of Boundary-Modeling AD and Alignment-Based AD.

decision criteria specify the region accepted by the pre-modeled boundary. Behavior patterns outside the accepted region are classified as anomalous. DeepLog and LogAnomaly learn normal log-sequence patterns [40, 123]. TraceAnomaly models normal invocation paths and response times [106], while OmniAnomaly and JumpStarter model normal multivariate service metrics [116, 164]. When system updates, workload changes, or configuration changes alter normal behavior, these learned patterns and their boundaries need to be updated. StepWise and LARA address such *boundary drift* by adapting detectors to the changed distributions [22, 117]. New runtime data can therefore support a transition from one relatively stable boundary to another. In LLM-based systems, however, historical normal patterns may not characterize expected behavior for the current request. An execution can fulfill the current request without resembling historical normal patterns. Conversely, an execution resembling those patterns can violate the current requirements. Normal and anomalous behavior patterns can consequently overlap, yielding a *chaotic boundary* when decision conditions are derived from historical runtime data alone. The difficulty then lies in determining whether the current behavior satisfies the applicable goals and constraints, not simply in updating historical normal patterns. Figure 4 compares this decision basis with Alignment-Based AD, and Section 3 reviews boundary-construction mechanisms.

2.2.2 Alignment-Based AD. Alignment-Based AD evaluates whether current behavior patterns satisfy the applicable requirements rather than relying on historical normal patterns alone. It constructs an *alignment model* that connects requirement instances with behavior patterns under the current execution context. The model establishes correspondences between these goals and constraints and the relevant entities, states, actions, or evidence. Its decision conditions specify what the behavior patterns must satisfy. The model can use graphs, matching rules, predicates, or logical properties. Evaluation can use learned scorers, pretrained models, symbolic checkers, or external verifiers. SLO Alignment, Intent Alignment, and Safety Alignment organize these checks according to the applicable goals and constraints. A retrieval component can violate an SLO by timing out, user intent by returning irrelevant evidence, or safety by accessing unauthorized data. Factuality checks align generated claims with the evidence available for the current request [25, 38, 122]. The distinction from Boundary-Modeling AD lies in the *principal decision basis*: an alignment-based method constructs the current requirement–behavior correspondences and decision conditions,

even when the checking algorithm is fixed. Methods that also apply a pre-modeled boundary combine the two paradigms. Figure 4 compares their decision bases, and Section 4 presents the five-stage framework for alignment-model construction, evaluation, and evolution.

2.3 Observable Data, Datasets, and Measurements

Observable data in LLM-based systems include traces, logs, and metrics. Traces link related execution steps, logs record execution events, and metrics quantify runtime behavior. *Agent traces* record task execution across model calls, tool invocations, memory updates, and interactions with the environment. They link actions with returned observations and subsequent actions, showing the sequence used to fulfill a user request [96, 104]. Structured spans attach prompts, responses, retrieved evidence, tool arguments and results, and policy decisions to these execution steps. By contrast, *reasoning traces* record the intermediate reasoning text generated by an LLM, such as deductions, calculations, and revisions leading to its answer [199]. One agent trace can contain reasoning traces from several model calls and associate them with the corresponding actions and observations. Runtime measurements include latency, errors, resource utilization, token usage, time to first token, entropy, and inference cost. Evaluation datasets specify the outputs or actions to be assessed and the corresponding anomaly labels. For hallucination detection, response-level labels support answer assessment, whereas claim- or token-level labels support localization [131]. For safety detection, NotInject assesses whether benign inputs are incorrectly blocked [94], while attack evaluations can measure guardrail misclassification [55] or harmful outcomes during subsequent execution. These evaluation targets distinguish benign-input false positives, attack success, and detector recall. Detection quality is measured using threshold-dependent accuracy, precision, recall, F1, and error rates, or score-based AUROC and AUPRC. Runtime evaluation also measures detection delay and overhead. Comparisons need to account for model versions, task distributions, evidence access, anomaly prevalence, attack settings, decision thresholds, and interventions. We therefore compare methods by their observation and verification needs rather than combine scores from different evaluation settings.

2.4 Related Research Domains

Related work on LLM-based systems is commonly described through hallucination detection, uncertainty and out-of-distribution detection, runtime verification, guardrails, and testing. These descriptions emphasize detection targets, evaluation techniques, or execution stages. This survey connects their anomaly detection methods through the principal decision basis used to characterize expected behavior. Methods that evaluate departures from pre-modeled statistical, structural, or feature regularities follow Boundary-Modeling AD. Methods that construct correspondences between current behavior patterns and applicable requirements follow Alignment-Based AD. Hallucination detection, for example, can use a learned anomaly boundary or check generated claims against evidence for the current request. The same detection target can therefore be addressed under either paradigm. Some studies instead organize anomalies by their *location* or the affected system layer. The seven-layer harness provides an architectural reference for locating anomalies in runtime components [96]. AgentOps groups anomalies by *type*, distinguishing reasoning, action, memory, security, task-specification, orchestration, communication, and termination anomalies [181]. A given location or type can be associated with different *requirement violations*. A tool call can breach a latency target, fail to fulfill user intent, or violate authorization. This motivates organizing Alignment-Based AD by SLO, user intent, and safety requirements, rather than by component or anomaly type alone. RecurGuard relates reasoning-trace growth to task progress to identify abnormal resource consumption [10]. VeriSafe Agent checks GUI actions against user instructions [89]. PlanGuard checks proposed tool calls against instruction-derived action references to

Table 1. Mapping anomaly locations and anomaly types to violations of expected behavior in LLM-based systems.

Anomaly location				Anomaly type			
Harness layer	**SLO**	**Intent**	**Safety**	**AgentOps type**	**SLO**	**Intent**	**Safety**
Execution environment	✓	✓		Reasoning	✓	✓	✓
Tool interface	✓	✓	✓	Action	✓	✓	✓
Context / memory		✓	✓	Memory		✓	✓
Lifecycle / orchestration	✓	✓		Security			✓
Observability / operations	✓		✓	Task specification	✓	✓	✓
Verification / evaluation	✓	✓	✓	Orchestration	✓	✓	
Governance / security		✓	✓	Communication	✓	✓	✓
				Termination	✓	✓	

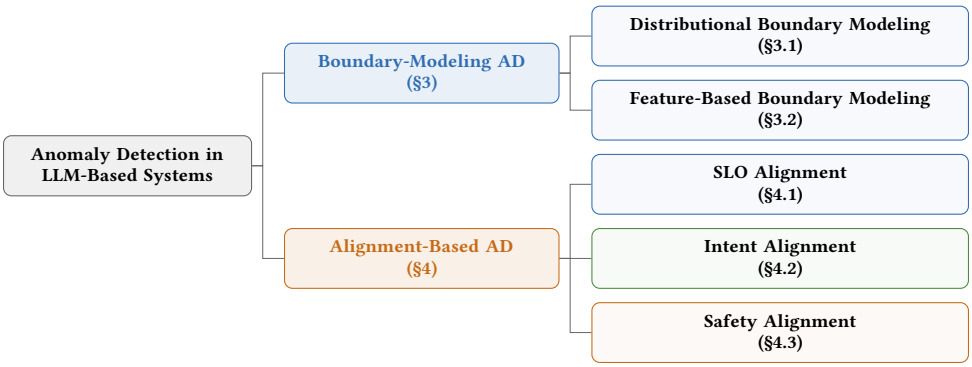


Fig. 5. Taxonomy of anomaly detection methods in LLM-based systems.

detect indirect prompt injection [52]. These decision conditions motivate their placement in SLO Alignment, Intent Alignment, and Safety Alignment, respectively. Table 1 relates anomaly locations and types to requirement violations, while Figure 5 groups methods by the two anomaly detection paradigms and their categories.

3 Boundary-Modeling AD

Boundary-Modeling AD follows prior work on anomaly detection in software systems. It constructs a boundary characterizing expected behavior by modeling statistical distributions, structural regularities, or anomaly-related features from historical runtime data. Normal or labeled observations support model learning and boundary calibration. Generated comparison samples can augment these observations, while predefined criteria guide boundary construction. The detector then compares current behavior patterns with the pre-modeled boundary to identify anomalies in outputs, reasoning steps, tool use, or trajectories. Let $\mathcal{D}_i^{\text{ref}}$ denote the reference observations available to model version i. Let ϕ map observable data $z_{0:t}$ to the representation used by a detector. Boundary construction can be written as

$$\theta_i = \mathcal{M}(\mathcal{D}_i^{\text{ref}}), \qquad B_i^{\text{pre}} = \left\{ z_{0:t} \mid s_{\theta_i}(\phi(z_{0:t})) \le \gamma_i \right\}, \tag{2}$$

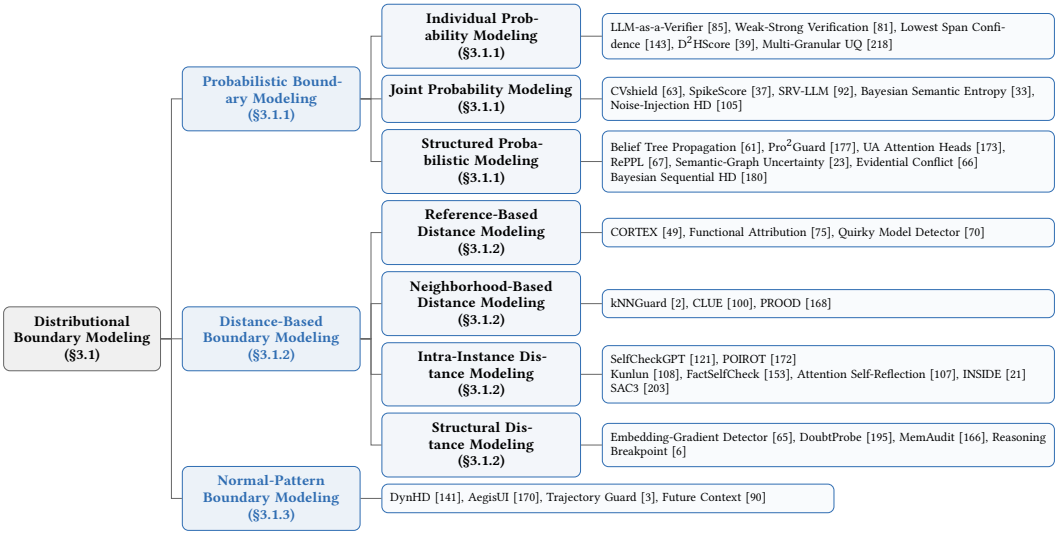


Fig. 6. Taxonomy of Distributional Boundary Modeling.

where $\mathcal{M}$ is the modeling procedure, θ_i is the resulting parameter set, s_{θ_i} is an anomaly score, and γ_i is a decision threshold. B_i^{pre} denotes the region accepted by the pre-modeled boundary. The detector then applies

$$A_t = \mathbb{I}\left[s_{\theta_i}(\phi(z_{0:t})) > \gamma_i\right]. \tag{3}$$

Methods differ in how they construct B_i^{pre}. *Distributional Boundary Modeling* uses probabilities, distances, or recurring normal patterns; *Feature-Based Boundary Modeling* uses anomaly-related features. Figures 6 and 7 map studies to their primary mechanisms.

3.1 Distributional Boundary Modeling

Distributional Boundary Modeling uses statistical or structural regularities in reference observations. For an output or execution representation $h = \phi(z_{0:t})$, probabilistic methods estimate likelihood, distance-based methods measure separation from a reference, and normal-pattern methods learn recurring sequences, structures, or reconstructions. Calibrated deviations determine anomaly decisions:

$$s_\theta(h) = \begin{cases} -\log p_\theta(h), & \text{probabilistic modeling,} \\ d(h, \mathcal{R}_\theta), & \text{distance-based modeling,} \\ d(h, g_\theta(h_{<t})), & \text{normal-pattern modeling,} \end{cases} \tag{4}$$

where $\mathcal{R}_\theta$ is a reference region. The final case illustrates temporal normal-pattern modeling: $h_{<t}$ denotes the representations of previous observations, and $g_\theta(h_{<t})$ predicts the current representation h. Structural and reconstruction methods instead compare h with a learned normal structure or its reconstruction. In LLM-based systems, these methods are used mainly to detect hallucinated outputs that violate the intent requirement, unsafe prompts or trajectories that violate the safety requirement, and operational deviations that violate the SLO requirement.

3.1.1 Probabilistic Boundary Modeling. Probabilistic methods use token probabilities, entropy, confidence estimates, or variation across generations. Likelihood, uncertainty, or predicted risk outside a calibrated region indicates an anomaly. The models concern individual observations, related observations jointly, or structured dependencies.

Individual Probability Modeling. Individual probability modeling commonly begins with token likelihoods, confidence scores, or local entropy statistics to identify outputs that deviate from expected behavior [85, 143]. LLM-as-a-Verifier uses model-generated confidence assessments as the decision reference, while Lowest Span Confidence identifies hallucinations from the least confident span within a response. D^2HScore models uncertainty patterns across token positions and reasoning steps [39]. Weak-Strong Verification incorporates verification strength into the anomaly decision, whereas Multi-Granular UQ combines token-, sequence-, temporal-, and distribution-level probability data [81, 218]. Although these methods use different probability granularities, they all pre-model an intent-related boundary from probability data associated with a single observed output or execution.

Joint Probability Modeling. Joint probability methods construct the boundary from probability variation across related observations. CVshield and Noise-Injection HD generate related reasoning or perturbed observations and evaluate their agreement under a calibrated probabilistic interpretation [63, 105]. SRV-LLM estimates runtime robustness across repeated observations, while Bayesian Semantic Entropy groups generations by meaning and allocates additional samples when uncertainty remains high [33, 92]. These methods differ in how they obtain related observations. SpikeScore induces post-answer, multi-turn continuations and scores abrupt uncertainty fluctuations across them [37]. Their decision criteria use variation across observations rather than a confidence value from one output.

Structured Probabilistic Modeling. Structured probabilistic methods represent dependencies among claims, reasoning steps, or execution states through trees, attention structures, and probabilistic transition systems. Belief Tree Propagation and Semantic-Graph Uncertainty propagate uncertainty across related claims or semantic nodes [23, 61]. UA Attention Heads, RePPL, and Evidential Conflict construct structured probability evidence from model states or internal relations [66, 67, 173]. Pro^2Guard and Bayesian Sequential HD instead model how risk evolves across execution states or sequential observations [177, 180]. Pro^2Guard evaluates execution risk by combining learned transition probabilities with formal safety properties [177].

3.1.2 Distance-Based Boundary Modeling. Distance-based methods avoid estimating a normalized probability distribution. They pre-model a reference geometry and identify anomalies from reference distance, neighborhood membership, intra-instance distance, or structural distance. Their effectiveness depends on whether the selected representation places observations that satisfy the applicable requirement close together and separates observations that violate it.

Reference-Based Distance Modeling. Reference-based methods compare observations with evidence, trusted outputs, or normal executions using a predefined or calibrated distance. CORTEX contrasts token representations with and without retrieved evidence, while Quirky Model Detector compares internal representations with a calibrated reference [49, 70]. Functional Attribution instead samples parameter space to measure coupling with trusted samples and detect anomalous mechanisms such as backdoors [75].

Neighborhood-Based Distance Modeling. Neighborhood-based methods compare observations with nearby samples, centroids, or density regions. kNNGuard uses stored safe and unsafe hidden activations; CLUE uses reasoning-trace centroids from prior executions; PROOD detects observations outside high-density regions [2, 100, 168].

Intra-Instance Distance Modeling. Intra-instance methods construct multiple related variants of the same input, output, or execution step, and model the boundary using distances among these variants. For the same input or output, variants can be constructed through repeated sampling, perturbation, reconstruction, or semantic grouping. SelfCheckGPT, FactSelfCheck, Attention Self-Reflection, INSIDE, and SAC3 use these strategies to construct and compare output variants [21, 107, 121, 153,

203]. For the same execution step, POIROT constructs variants of execution evidence, while Kunlun obtains kernel-trace variants from synchronized GPU workers [108, 172].
Structural Distance Modeling. Structural distance methods transform observable data into geometric or relational structures and model the boundary using distances or deviations within these structures. Embedding-Gradient Detector represents conditional and unconditional outputs in the output probability space and approximates their distributional distance using embeddings and first-order gradients. DoubtProbe constructs structured representations of the original and reconstructed requests and measures whether their information is preserved. MemAudit builds a memory-consistency graph and identifies memories whose relations differ from the surrounding memory structure. Reasoning Breakpoint represents reasoning steps as a hidden-state trajectory and detects the step whose transport cost departs from the stable trajectory geometry [6, 65, 166, 195].

3.1.3 Normal-Pattern Boundary Modeling. Normal-pattern methods construct normal patterns by learning recurring sequences, reconstruction regularities, or stable execution structures from reference data. These normal patterns form the pre-modeled boundary of expected behavior. At the output level, Future Context exploits the tendency of hallucinated content to persist in sampled continuations, while DynHD learns the expected evolution of uncertainty during diffusion decoding and detects deviations from that trajectory [90, 141]. At the agent level, AegisUI uses a benign-trained autoencoder to model the reconstruction pattern of structured UI payloads, and Trajectory Guard combines contrastive learning with a recurrent autoencoder to model sequential validity in agent trajectories [3, 170]. Although these methods operate on different observable data, each constructs its boundary from a recurring pattern that can be learned before the monitored execution.

3.2 Feature-Based Boundary Modeling

While Distributional Boundary Modeling constructs the boundary from probability distributions, distances, or normal patterns, another line of work first represents observable data through features associated with anomalies. Feature-Based Boundary Modeling uses these features to pre-model the boundary of expected behavior. The features may be explicitly defined, extracted from pretrained models, learned for a particular detection task, mapped from predefined requirements, or updated through adaptation. For methods trained with labeled reference data $\mathcal{D}^{\text{ref}} = \{(z_j, y_j)\}_{j=1}^{n}$, feature learning can be written as

$$h_j = \phi_\eta(z_j), \qquad (\eta^*, \theta^*) = \arg\min_{\eta,\theta} \sum_{j=1}^{n} \ell\big(f_\theta(\phi_\eta(z_j)), y_j\big), \tag{5}$$

where ϕ_η maps observable data to anomaly-related features and f_θ learns the decision boundary from these features. Existing methods differ mainly in how ϕ_η is constructed. Some methods model features through predefined transformations, while others use pretrained models or task-specific training. Additional methods map requirements into features or update feature models using newly accumulated data. We therefore organize Feature-Based Boundary Modeling according to these feature-construction methods.

3.2.1 Predefined Feature Modeling. Predefined feature modeling applies fixed transformations to observable data. Features are scored directly or combined by a lightweight classifier. Explicit definitions aid interpretation, while inference cost depends on the transformations and detector.
Metric-Based Feature Construction. Metric-based methods use predefined numerical measurements. Entropy Production Rate and Diversion Decoding measure token probabilities, entropy, and resistance to alternative decoding; VERDI combines decomposed verification measurements [1, 128, 140]. Sanity Checks summarizes lexical, reasoning, interaction, and timing behavior [215].

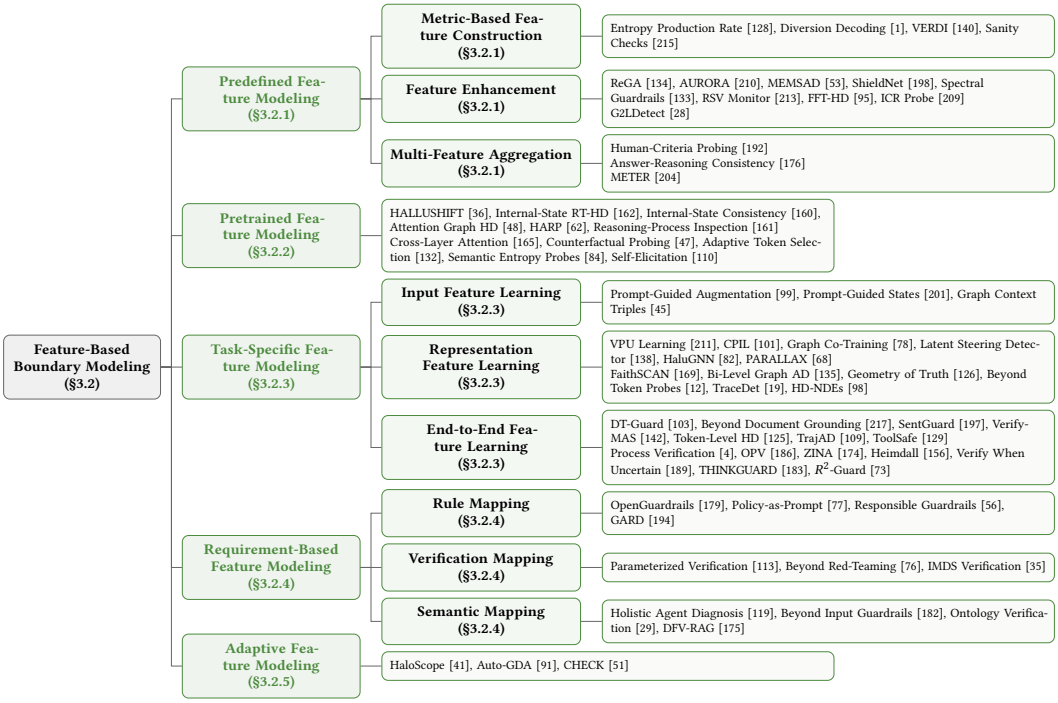


Fig. 7. Taxonomy of Feature-Based Boundary Modeling.

Feature Enhancement. Feature enhancement transforms or augments observations before boundary construction. ReGA and AURORA construct graph or relational features, while FFT-HD and Spectral Guardrails use spectral representations of temporal or internal-model data [95, 133, 134, 210]. MEMSAD, ShieldNet, RSV Monitor, ICR Probe, and G2LDetect augment, expand, or convert representations to expose anomaly-related information [28, 53, 198, 209, 213].
Multi-Feature Aggregation. Multi-feature aggregation combines complementary signals into one decision. Human-Criteria Probing (HCPD) learns weights for interpretable truthfulness criteria; Answer-Reasoning Consistency (RACE) jointly evaluates reasoning consistency, answer uncertainty, answer–reasoning alignment, and coherence; METER ensembles tool-supported reasoning for multimodal hallucination detection [176, 192, 204]. Aggregating these signals broadens the information used for detection. The resulting boundary still depends on whether the predefined features distinguish normal and anomalous behavior in the target model, task, and domain.

3.2.2 Pretrained Feature Modeling. Pretrained LLMs, encoders, or multimodal models extract embeddings, hidden states, attention relations, or internal subspaces. The extractor can remain fixed while a downstream detector or threshold is calibrated. HALLUSHIFT and Internal-State RT-HD use hidden-representation changes, whereas Attention Graph HD and Cross-Layer Attention use attention-head or layer relations [36, 48, 162, 165]. HARP, Internal-State Consistency, Counterfactual Probing, Adaptive Token Selection, and Semantic Entropy Probes extract reasoning subspaces or token-level features [47, 62, 84, 132, 160]. Prompting enriches these representations: Reasoning-Process Inspection extracts embeddings during statement verification, while Self-Elicitation incorporates prior statements and organizes encoded knowledge into a graph [110, 161].

3.2.3 Task-Specific Feature Modeling. Task-specific feature modeling uses labels, contrastive samples, verification feedback, or detection objectives to learn anomaly-relevant distinctions. Training can shape input data, feature representations, or the complete detector, yielding input, representation, and end-to-end feature learning.

Input Feature Learning. Input feature learning constructs task-specific inputs before feature extraction. Prompt-Guided Augmentation generates truthful and hallucinated responses as training data [99]. Prompt-Guided States uses prompts to make truthfulness-related information more salient in the model's internal states [201]. Graph Context Triples decomposes generated text into knowledge triples and organizes them as a contextual graph before feature learning [45].

Representation Feature Learning. Representation feature learning directly optimizes how observable data are encoded in the feature space. VPU Learning and FaithSCAN integrate visual, uncertainty, and cross-modal information [169, 211]. Graph Co-Training, HaluGNN, and Bi-Level Graph AD learn graph representations of semantic or agent relations [78, 82, 135]. Latent Steering Detector, PARALLAX, Beyond Token Probes, TraceDet, and HD-NDEs learn from hidden-state or generation dynamics [12, 19, 68, 98, 138]. CPIL and Geometry of Truth use contrastive objectives to organize faithful and anomalous data in the learned feature space [101, 126].

End-to-End Feature Learning. End-to-end feature learning jointly trains the feature extractor and anomaly detector using labels, verification feedback, or rewards. DT-Guard, SentGuard, THINK-GUARD, and R^2-Guard learn safety decisions from labeled or reasoning-augmented data [73, 103, 183, 197]. Beyond Document Grounding, Token-Level HD, and ZINA train response- or span-level hallucination detectors [125, 174, 217]. ToolSafe learns step-level tool-safety decisions, while Verify When Uncertain learns when additional verification is required [129, 189]. VerifyMAS, TrajAD, Process Verification, OPV, and Heimdall train verifiers from trajectory supervision, expert feedback, or rewards [4, 109, 142, 156, 186]. Task-specific feature modeling can closely align the learned representation with a target anomaly-detection objective. Its boundary, however, depends on the tasks, models, labels, and feedback represented in the reference data.

3.2.4 Requirement-Based Feature Modeling. Requirement-based feature modeling maps requirements and observable data into a common evaluation representation. The resulting features indicate whether the observed behavior satisfies or violates a predefined SLO, intent, or safety requirement. Rules, verification procedures, or semantic criteria define how observable data are mapped to requirement-related features. These features provide the representation on which the detector applies its pre-modeled boundary. Accordingly, we organize these methods into rule mapping, verification mapping, and semantic mapping.

Rule Mapping. Rule mapping represents requirements as policies, categories, or thresholds and maps observable data into rule-satisfaction or rule-violation features. OpenGuardrails uses configurable safety categories and sensitivity thresholds [179]. Policy-as-Prompt converts governance documents into source-linked policy trees and prompt-based classifiers [77]. Responsible Guardrails combines a symbolic topic classifier, toxicity measurements, and topic-sensitive thresholds [56]. GARD maps inputs and outputs to sensitive-financial- information categories derived from a domain taxonomy [194]. These rule-derived features pre-model the safety or SLO boundary used during detection.

Verification Mapping. Verification mapping represents requirements as formal properties, executable tests, state-transition models, or runtime monitors. The observed execution is then mapped into property-satisfaction or verification results. Parameterized Verification instantiates formal constraints for a selected configuration [113]. Beyond Red-Teaming converts safety requirements into executable checks [76]. IMDS Verification maps agent states and messages to a Petri net for

reachability, deadlock, and termination checks [35]. These methods provide property-satisfaction features that the detector evaluates against a pre-modeled boundary.

Semantic Mapping. Semantic mapping is used when a requirement cannot be expressed completely by rules or formal properties. It maps requirements and observable data into a shared semantic representation and derives alignment or deviation features. Holistic Agent Diagnosis maps span-level assessments to an agent-level diagnosis [119]. Beyond Input Guardrails compares cross-agent semantic flows with expected data-flow, control-flow, and intent relations [182]. Ontology Verification links extracted candidates to canonical entities and relations before applying ontology constraints [29]. DFV-RAG maps generated answers into semantic-matching and factual-correctness features [175]. The detector evaluates these semantic features against a pre-modeled boundary.

3.2.5 Adaptive Feature Modeling. Feature boundaries can degrade when the model, task, domain, or policy changes. Adaptive methods use newly accumulated, generated, or target-domain data to revise the detector for later observations. Let $\Delta\mathcal{D}_i$ denote the new reference data. The update is

$$\theta_{i+1} = \mathcal{U}(\theta_i, \Delta\mathcal{D}_i), \qquad B_i^{\text{pre}} \longrightarrow B_{i+1}^{\text{pre}}. \tag{6}$$

Adaptive methods update the reference data, feature extractor, classifier, or policy representation used by the detector. HaloScope estimates truthful and hallucinated mixtures from unlabeled generations collected after deployment and retrains a truthfulness classifier [41]. Auto-GDA iteratively generates target-domain samples, assigns weak labels with teacher models, and uses the selected samples to adapt a grounding verifier [91]. CHECK continuously updates a clinical hallucination classifier using structured medical evidence [51]. The updated classifiers evaluate later observations.

Uncertainty in LLM-generated outputs and action choices makes expected behavior difficult to characterize through historical runtime data alone. The same request can lead to different reasoning paths, evidence selections, and tool calls that fulfill user intent, while similar output formats or execution structures can conceal requirement violations. These variations weaken the correspondence between historical normal patterns and expected behavior, allowing valid and anomalous behavior patterns to overlap in a chaotic boundary. Changes in models, tasks, and domains also affect the transfer of learned boundaries. HALLUSHIFT studies distribution shift [36]. Cross-lingual hallucination detection examines transfer across languages [59]. SpikeScore reports poor transfer for several baselines and improves cross-domain detection using uncertainty fluctuations [37]. Adaptive updates use new runtime or target-domain data to revise reference data, feature representations, or detector parameters. They address boundary drift by moving from one relatively stable pre-modeled boundary to another. The decision conditions still derive from learned regularities, so this adaptation leaves unresolved the chaotic boundary in which valid and anomalous behavior patterns overlap. Alignment-Based AD addresses this limitation by constructing decision conditions that relate current behavior patterns to applicable requirements under the execution context, as examined next.

4 Alignment-Based AD

Alignment-Based AD establishes decision conditions by aligning current behavior patterns with requirements that express expected behavior. SLO, user intent, and safety requirements express the goals and constraints of system operation, task completion, and safe execution, respectively. Requirements may be expressed as natural-language instructions, rules, or logical conditions, whereas behavior patterns represent relevant states, actions, outputs, or relations modeled from runtime data. The alignment model connects these representations through correspondences and decision conditions under the current execution context. Methods evaluate the conditions to detect anomalies and update the model as the requirements, runtime data, or context change. Figure 8

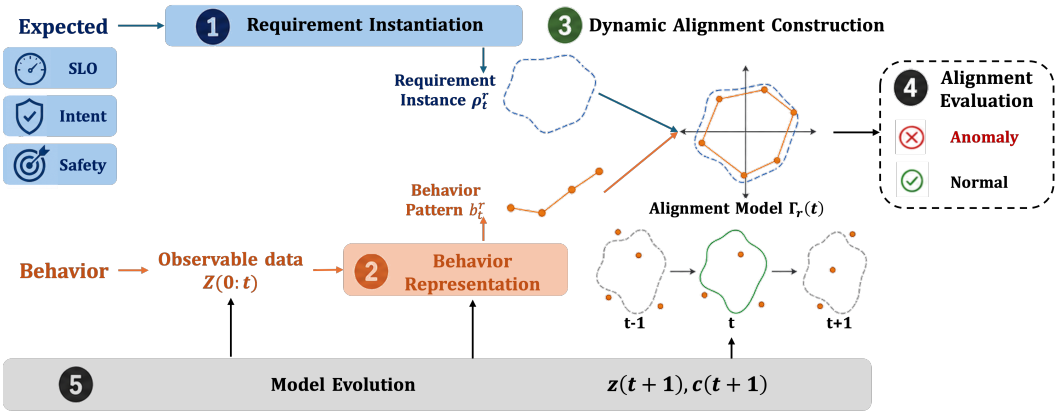


Fig. 8. Five-stage framework for Alignment-Based AD.

provides a five-stage framework for analyzing how alignment models are constructed, evaluated, and updated. Requirement Instantiation expresses goals and constraints in a form that can be assessed in the current execution. Behavior Representation organizes relevant runtime data and context into a behavior pattern. Dynamic Alignment Construction establishes correspondences and decision conditions. Alignment Evaluation checks whether the pattern satisfies these conditions and thus conforms to expected behavior [17]. Model Evolution updates the model as the information used for construction and evaluation changes. These activities may be implemented together.

Constructing decision conditions requires relating the applicable goals and constraints to the behavior being evaluated. An SLO requirement can specify a latency threshold or cost budget, but resource use may also need to be interpreted in relation to task progress. User intent introduces goals and constraints whose satisfaction depends on the current task and supporting evidence. Safety conditions depend on applicable policies, permissions, and environment states. A reasoning path or tool call can satisfy these conditions without resembling historical normal patterns, while historical similarity can coexist with a requirement violation. The following framework describes how an alignment model makes these requirement–behavior relations checkable and maintains them during execution.

To formalize this process, we consider a requirement dimension $r \in \{\text{SLO}, \text{intent}, \text{safety}\}$. For requirement dimension r, R^r denotes the general end-to-end requirement, q denotes the current request, $c_{0:t}$ denotes the execution context available at time t, and $z_{0:t}$ denotes the observable data collected up to time t.

Requirement Instantiation. Requirement instantiation converts a general end-to-end requirement into a requirement instance using the current request and execution context. An SLO may be instantiated as a latency, cost, or progress target. User intent may be instantiated as goals, constraints, or completion conditions for the current request. A safety requirement may be instantiated as an active policy, authorization condition, or state property. We represent this stage as

$$\rho_t^r = \mathcal{I}_r(R^r, q, c_{0:t}), \tag{7}$$

where ρ_t^r is the requirement instance at time t.

Behavior Representation. Behavior representation organizes observable data and execution context into a behavior pattern relevant to the current requirement instance. Observable data may include metrics, claims and supporting evidence, reasoning steps, tool calls, state transitions, policy decisions, and execution records. A behavior-representation function Φ_r uses the requirement

instance to organize relevant observable data and execution context into a behavior pattern.

$$b_t^r = \Phi_r(z_{0:t}, c_{0:t}; \rho_t^r), \tag{8}$$

Here, b_t^r represents the behavior relevant to the current requirement instance. Its construction uses observable data together with the requirement instance and execution context. It can describe the current execution without being a recurring normal pattern learned from historical data.

Dynamic Alignment Construction. Dynamic alignment construction constructs the correspondences and decision conditions used for detection. A modeling function $\mathcal{M}_r$ constructs the alignment model between the requirement instance and the behavior pattern under the current context:

$$\Gamma_t^r = \mathcal{M}_r(\rho_t^r, b_t^r, c_{0:t}). \tag{9}$$

Existing methods construct the alignment model through different mechanisms. Some convert the requirement instance into a target, obligation, policy, property, or oracle that can be applied to the behavior pattern. Others derive a relation from evidence, states, trajectories, or integrity records contained in the behavior pattern. A third group jointly represents the requirement instance and behavior pattern. In each case, Γ_t^r specifies how the behaviors and evidence correspond to the requirement instance and which decision conditions apply. It may express whether progress meets an operational target, whether evidence supports a claim, whether an artifact satisfies an obligation, whether an action is permitted in the current state, or whether an execution record supports its claimed authorization.

Alignment Evaluation. Alignment evaluation checks whether the behavior pattern satisfies the decision conditions in the alignment model. Score-based evaluation produces a satisfaction score s_t^r. A score below the threshold δ_r indicates an anomaly:

$$s_t^r = \mathcal{E}_r(\rho_t^r, b_t^r; \Gamma_t^r), \qquad A_t^r = \mathbb{I}\left[s_t^r < \delta_r\right], \tag{10}$$

where $A_t^r = 1$ denotes an anomaly under requirement dimension r. Boolean checks identify violations directly, setting $A_t^r = 1$ for a confirmed violation. Some conditions require later observations before they can be decided. Goal-completion obligations, for example, may remain pending until termination or a deadline. AgentLTL defers prefix obligations that cannot yet be decided [42]. Unavailable evidence likewise yields an unresolved result.

For requirement instances with multiple conditions, evaluation retains separate verdicts and supporting evidence for each condition. A violation of a mandatory constraint constitutes an anomaly even if unrelated goals are satisfied. Condition identifiers, violation locations, and requirement-instance and alignment-model versions make these results traceable.

Model Evolution. Model evolution maintains the decision conditions as requirements, runtime data, and execution context change. An updated requirement or new context may produce a new requirement instance. New runtime data may produce a new behavior pattern. Retrieved evidence may change which claims have sufficient support. A tool result may change the permitted next action, and a policy decision may change the agent's permissions. The alignment model is updated accordingly:

$$\Gamma_{t+1}^r = \mathcal{U}_r(\Gamma_t^r, \rho_{t+1}^r, b_{t+1}^r, c_{0:t+1}). \tag{11}$$

The updated model supplies decision conditions for evaluating subsequent behavior patterns. Earlier verdicts remain associated with the requirement instances, evidence, and model versions used at the time of evaluation, with reassessment and behavior repair recorded separately. For example, an authorization change governs subsequent actions, while earlier actions are assessed against the permissions active when they occurred. Existing methods update the alignment model in several ways. Tool-Integrated Verification retrieves new evidence while evaluating a medical reasoning trace. Each retrieval result updates the alignment model used to evaluate the remaining reasoning

steps [202]. CIVeX, Near-Miss, Control-Theoretic Guardrails, and SentinelAgent update a trajectory, control, or trust state after new events and actions. The updated state changes which actions remain consistent with the safety requirement [58, 137, 145, 150]. Other methods use verification feedback. Proactive Correction revises its verification strategy after a failure, while PACE and CoT Program Verification revise the behavior, the requirement instance, or both after a failed check [11, 57, 152].

We organize alignment methods first by the SLO, user intent, and safety requirement dimensions. Each dimension introduces goals and constraints that must be connected to the relevant behavior patterns. Figures 9 and 10 group methods by their principal alignment-model construction mechanism. Intent Alignment includes Constraint, Evidence, Reference, and Transformation Alignment. Safety Alignment includes Policy, Contract, Provenance, and Action Alignment. The following subsections use the five-stage framework to explain methodological contributions to these mechanisms. For example, AgentLTL expresses procedural goals and constraints as temporal-logic properties and evaluates them over agent events and trace prefixes [42]. SLO Alignment is discussed without further subdivision, and Model Evolution applies across all three requirement dimensions.

4.1 SLO Alignment

SLO Alignment evaluates behavior patterns against performance targets, resource budgets, and reproducibility constraints specified by SLO requirements. For a given requirement instance, the alignment model relates latency, cost, and resource measurements to the applicable targets and budgets. In LLM-based systems, the reasoning path and tool calls needed to fulfill a request are not fixed in advance. Resource counts alone do not establish whether additional computation contributes to task completion. Evaluating reasoning-token consumption therefore also involves relating resource use to task progress. RecurGuard constructs this relation using repetition, reasoning-trace growth, and semantic progress toward the user query. Its decision condition identifies abnormal token consumption when the reasoning trace continues to grow without sufficient progress [10]. MarginGate addresses reproducibility at the Alignment Evaluation stage. It uses the logit margin at each decoding step to identify steps at risk of selecting a different token under a changed batch context. It then verifies these steps against the reproducibility requirement [32]. LLM Readiness Harness also emphasizes Alignment Evaluation. It evaluates task outcomes, groundedness, cost, and latency against the SLO requirements of the deployment scenario [120].

4.2 Intent Alignment

Intent Alignment evaluates whether behavior patterns achieve the outcome requested by the user and satisfy the constraints on how it is achieved. Interpreting a natural-language request requires identifying its task goal, constraints, and completion conditions. A fluent response or a plausible sequence of actions does not by itself establish that the request has been fulfilled. The relevant checks relate generated claims to supporting evidence, actions to requested state changes, or final artifacts to task-specific correctness conditions. The alignment model makes these relations explicit using the current requirement instance and relevant runtime data. Constraint Alignment expresses instruction-derived conditions, Evidence Alignment relates claims to supporting source data, Reference Alignment compares behavior with a task-specific reference, and Transformation Alignment checks relations that should hold across transformed inputs or outputs. The following subsections analyze how these mechanisms instantiate requirements, represent behavior, construct alignment models, and evaluate goal achievement and constraint satisfaction.

4.2.1 Requirement Instantiation. Some methods address the difficulty of checking natural-language user intent at the Requirement Instantiation stage. They identify its goals, constraints, and completion conditions and express them as conditions that can be evaluated against current behavior.

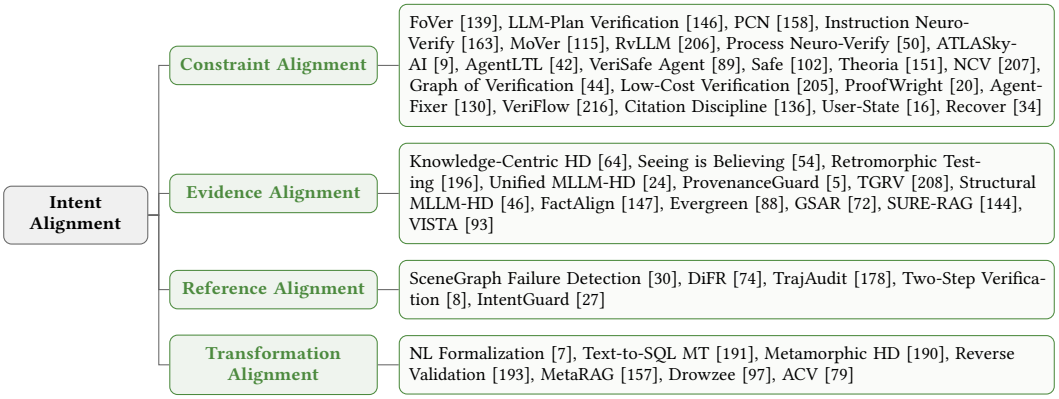


Fig. 9. Taxonomy of Intent Alignment.

Specification instantiation expresses these conditions directly. Decomposition instantiation distributes them across execution steps. Validation instantiation connects them to a concrete check. These designs provide a checkable requirement instance for constructing the alignment model.

Specification Instantiation. Specification instantiation expresses user intent as predicates, logical constraints, or temporal properties. FoVer checks first-order formalizations of reasoning with a theorem prover, while LLM-Plan Verification represents plans as Kripke structures with linear-temporal-logic properties [139, 146]. MoVer and RvLLM provide spatial/temporal and domain-specific predicates, respectively [115, 206]. Instruction Neuro-Verify uses constraint satisfaction; PCN binds generated numbers to structured claims and verification policies [158, 163]. Process Neuro-Verify applies physical and causal constraints to process-control explanations, whereas ATLASky-AI combines ontology, standards, and physical constraints for spatiotemporal knowledge [9, 50].

Decomposition Instantiation. Decomposition instantiation organizes an end-to-end requirement across reasoning steps, workflow stages, or trajectory prefixes. It specifies what the current behavior must satisfy at each stage of an execution. AgentLTL expresses procedural requirements over agent traces and checks completed traces or individual trace prefixes [42]. VeriSafe Agent translates a user instruction into a formal specification and checks each GUI action before it is executed [89]. Safe converts the claims made at each mathematical reasoning step into Lean statements, while Theoria rewrites a solution as typed state transitions with an explicit justification for every transition [102, 151]. VISTA decomposes dialogue responses into factual claims and tracks their consistency across turns [93]. NCV and Graph of Verification organize reasoning into verification nodes. NCV uses node-wise binary checks, whereas Graph of Verification adjusts node granularity to the structure of the reasoning process [44, 207]. Low-Cost Verification decomposes multimodal outputs into events and uses verifier feedback to locate inconsistent event information [205]. The active condition can therefore change as the execution advances while remaining linked to the end-to-end user intent.

Validation Instantiation. Validation instantiation defines how current behavior will be checked against user intent. NL Formalization converts a natural-language policy into logical form and cross-checks multiple formalizations for semantic equivalence [7]. Text-to-SQL MT defines metamorphic relations for schema linking and logical synthesis. It checks whether source and follow-up SQL outputs preserve these relations [191]. ProofWright uses formal safety properties and semantic equivalence to validate generated CUDA kernels [20]. AgentFixer constructs task-specific checks for failures in agent inputs, plans, and outputs [130]. VeriFlow combines workflow graph checks, semantic constraints, and sandbox execution to validate generated workflows [216]. Citation

Discipline requires generated code to cite the requirement associated with each line. This relation makes unsupported code detectable across the specification and implementation artifacts [136].

4.2.2 Behavior Representation. Some methods address the difficulty of relating observable data to user intent at the Behavior Representation stage. They organize generated content and relevant source data so that the conditions in user intent can be evaluated against a behavior pattern. Source data may include task evidence, retrieved documents, database records, knowledge graphs, images, dialogue history, and tool results. These methods identify factual or task-related statements in generated content as claims. Alignment Representation associates each claim with relevant source data. Structural Representation organizes their entities and relations. Aggregation Representation combines local evidence for the complete response. The resulting behavior pattern makes support, conflict, coverage, and provenance available for constructing the alignment model with the requirement instance.

Alignment Representation. Alignment representation constructs local claim–source relations that expose support or conflict. Knowledge-Centric HD aligns claim triples with references, Seeing is Believing aligns descriptions with pixel-level image regions, and Retromorphic Testing aligns RAG claims with supporting or conflicting context spans [54, 64, 196]. Unified MLLM-HD selects auxiliary tools and relates multimodal claims to their outputs [24]. ProvenanceGuard additionally records tool IDs, source IDs, and raw Model Context Protocol (MCP) tool outputs to preserve source attribution alongside factual support [5].

Structural Representation. Structural representation preserves dependencies within and between claims and sources through graphs, connected triples, or semantic queries. TGRV aligns and reconstructs response and external-knowledge graphs; FactAlign aligns graphs of generated and reference facts [147, 208]. Structural MLLM-HD represents fine-grained differences between visual and textual scene graphs [46]. Evergreen instead turns claims into database verification queries and records supporting tuples [88].

Aggregation Representation. Aggregation representation combines local relations into response-level support and conflict information. GSAR aggregates claim grounding types and source weights with conflict penalties; SURE-RAG aggregates claim–passage coverage, relation strength, disagreement, conflict, and retrieval uncertainty [72, 144]. Unlike structural representation, the emphasis is the combined response-level result rather than dependencies among claims and sources.

4.2.3 Dynamic Alignment Construction. Methods construct requirement–behavior correspondences through a shared state representation or a task-specific reference. Shared-State Alignment Construction represents the goals and constraints in the requirement instance together with the relevant states, claims, or actions in the behavior pattern. Reference-Mediated Alignment Construction obtains or generates a reference for the current execution and relates the behavior pattern to it. These designs support the Constraint, Evidence, and Reference Alignment mechanisms by making their decision conditions checkable.

Shared-State Alignment Construction. Shared-state methods construct a state representation that contains both the goals or constraints from the requirement instance and the claims, actions, or state changes from the behavior pattern. They construct the alignment model within this shared representation. User-State reconstructs the transactional state of a user from the event history. It binds the agent's claims to the same state and checks whether they satisfy its invariants [16]. SceneGraph Failure Detection represents the state required by the current task step and the state observed in multi-view images as scene graphs. It constructs the alignment model from their structural correspondence [30]. Recover represents the task requirement and the current environment with ontology predicates. Logical rules relate these predicates and identify behavior that requires recovery [34].

Reference-Mediated Alignment Construction. Reference-mediated methods obtain or generate a verification reference that connects the requirement instance with the behavior pattern. DiFR uses a trusted implementation to produce reference predictions under a synchronized sampling seed. It relates generated tokens or activation fingerprints to these predictions [74]. TrajAudit uses the current test-failure report as a diagnostic reference. An investigator relates this reference to the relevant parts of the coding-agent trace [178]. Two-Step Verification first generates a prior that describes the desired behavior. It then relates the current trajectory to this prior [8]. IntentGuard uses the trusted user request and required context as a verification reference. It relates the selected tool and its parameters to this reference [27].

4.2.4 Alignment Evaluation. Alignment Evaluation checks whether original and transformed outputs satisfy the expected relations. It also verifies whether reverse queries recover the original entities. Metamorphic HD generates follow-up questions whose answers should be preserved by synonym mutations and reversed by antonym mutations, aggregating violations into a hallucination score [190]. MetaRAG applies this principle to RAG factoids: retrieved context should entail synonym variants and contradict antonym variants [157]. Drowzee instead creates fact-conflicting cases through natural-language and logical transformations. Its semantic oracle specifies the expected relation between original and transformed outputs, and violations indicate hallucination [97].

Reverse Validation evaluates whether the factual information in a generated passage can recover its subject. It removes an entity from the passage and uses the remaining information to construct a reverse query. The returned entity should match the removed entity. A mismatch indicates that the passage is inconsistent with the factuality requirement [193]. ACV performs the evaluation at the claim level. It decomposes generated content and source documents into atomic claims and represents each claim as a knowledge-graph triple. It then creates entity, relation, temporal, and quantitative variants for pairwise verification. The claim-level results are aggregated into a verdict for the complete response [79].

4.3 Safety Alignment

Safety Alignment evaluates whether behavior patterns satisfy the policies, permissions, and obligations governing safe execution. Its decision conditions address authorized information flows and tool use, capability containment, and execution integrity. A behavior pattern that fulfills user intent does not necessarily satisfy these conditions. Whether an action is permitted depends on its authorization, the active policy, and the execution state. The alignment model connects current behavior with applicable safety conditions using authorization records, execution provenance, and state information. Policy Alignment checks behavior against active policy conditions. Contract Alignment checks capability and containment obligations. Provenance Alignment establishes whether observed behavior is supported by the claimed authority and execution record. Action Alignment evaluates proposed actions and their effects under the current state. Changes in permissions, policies, or state require the corresponding decision conditions to be updated. The following subsections analyze how these mechanisms instantiate safety requirements, represent the required execution information, and construct alignment models under the current execution context.

4.3.1 Requirement Instantiation. Requirement Instantiation determines the safety goals and constraints applicable to the current execution. Policy Alignment can express these conditions as natural-language clauses, checkable rules, or properties, while Contract Alignment expresses them as contracts or proof obligations. Semantic, Executable, and Contract Instantiation describe how these requirement instances are prepared for comparison with the current behavior pattern.
Semantic Instantiation. Semantic instantiation preserves the applicable safety policy as natural-language clauses. A model evaluates current behavior against these clauses. SingGuard receives

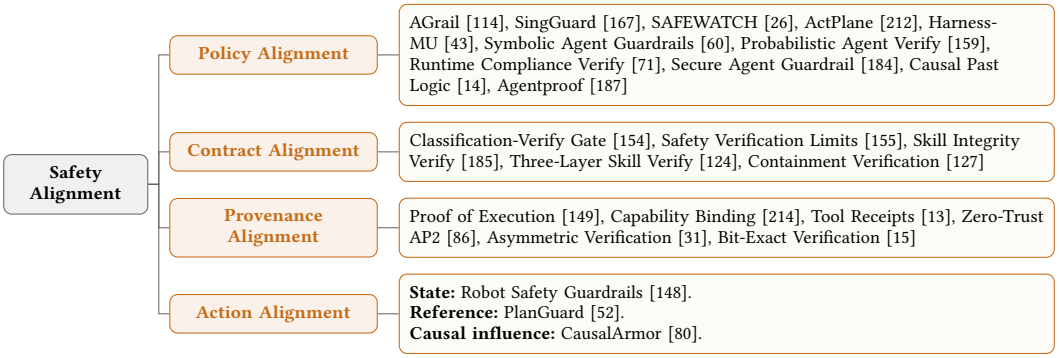


Fig. 10. Taxonomy of Safety Alignment.

the active policy at runtime and checks multimodal behavior clause by clause. SAFEWATCH encodes the safety policy applicable to the current video and evaluates the video against each policy component [26, 167].

Executable Instantiation. Executable instantiation converts the safety requirement into a rule, predicate, temporal formula, guard, or automaton. The resulting requirement instance can be checked or enforced over current behavior. ActPlane compiles cross-event policies into information-flow rules enforced at the operating-system boundary. Harness-MU represents permissions and authority order as execution hooks. Symbolic Agent Guardrails converts domain safety requirements into symbolic checks over inputs, outputs, and actions [43, 60, 212]. AGrail generates and revises safety checks using the current request, proposed action, and environment, then executes these checks to gate that action [114]. It retains reusable safety checks across tasks to support subsequent check construction. Probabilistic Agent Verify expresses security policies as probabilistic predicates. Runtime Compliance Verify converts regulatory requirements into predicates over agent traces. Secure Agent Guardrail and Causal Past Logic express security and causal requirements as runtime properties. Agentproof compiles temporal safety policies into automata over agent workflow graphs [14, 71, 159, 184, 187].

Contract Instantiation. Contract instantiation expresses the safety requirement as a contract or proof obligation over a system capability or boundary. Classification-Verify Gate and Safety Verification Limits construct verification conditions for safety claims. Skill Integrity Verify compares declared and implemented capabilities. Three-Layer Skill Verify constructs capability-containment obligations from a skill manifest. Containment Verification specifies the boundary properties that an agent framework must enforce [124, 127, 154, 155, 185].

4.3.2 Behavior Representation. Behavior Representation organizes authorization, provenance, and execution records needed to evaluate the applicable safety conditions. These records distinguish actions that appear similar but differ in authorization or execution integrity. In Provenance Alignment, Observability-Enriched Representation adds the relevant information during data generation. Execution-Reconstruction Representation reconstructs or reproduces the execution from collected runtime data.

Observability-Enriched Representation. Observability-enriched methods record authority, execution paths, results, and context that action records alone omit. Proof of Execution records contracts, causal events, and replay context; Capability Binding adds tool-invocation capabilities and interaction chains [149, 214]. Tool Receipts generates signed execution receipts containing the tool name, input and output hashes, and a timestamp, while Zero-Trust AP2 records a payment's mandate,

nonce, and runtime context [13, 86]. These records distinguish superficially identical actions with different authority or integrity and support alignment with applicable safety requirements.
Execution-Reconstruction Representation. Execution reconstruction exposes the computation behind an output for execution-integrity checks. Asymmetric Verification records information for deterministic replication and partial auditing; Bit-Exact Verification preserves the software, hardware, and numerical conditions for exact inference reproduction [15, 31]. The reconstructed behavior pattern is then related to the applicable safety requirement.

4.3.3 Dynamic Alignment Construction. Some methods address the dependence on external conditions at the Dynamic Alignment Construction stage. They combine the requirement instance and the behavior pattern with the current environment state and execution context. Their main design determines the conditions under which current behavior satisfies the safety requirement. They construct the alignment model through safe-set membership, request–plan consistency, or causal influence. These designs allow the same behavior to receive different safety judgments under different conditions. Safety Guardrails for LLM-Enabled Robots uses the current physical state to construct the safe set specified by the safety requirement. It represents the alignment model as membership in this safe set. A proposed robot action satisfies the decision conditions only when it remains inside the safe set. Because the physical state changes during execution, the safe set and the alignment model also change [148]. PlanGuard uses an isolated planner to derive valid actions from trusted user instructions. It checks proposed tool calls against this reference, first applying hard constraints and then verifying whether parameter deviations preserve the user's intent. This alignment model connects the instruction-derived action requirements with current tool-use behavior to detect indirect prompt injection [52]. CausalArmor compares the influence of untrusted spans and the user request on a proposed privileged action using normalized leave-one-out attribution. It applies a threshold to the attribution difference to select spans for sanitization [80]. The attribution difference serves as a proxy for prompt-injection risk. These methods establish decision conditions for Action Alignment through safe-set membership, request–plan consistency, or causal influence. Each alignment model is constructed from information available during the current execution and can change as that information changes.

Alignment-Based AD connects current behavior patterns to the goals and constraints of SLO, user intent, and safety requirements under the execution context. The reviewed methods establish checkable decision conditions using task-specific properties, supporting evidence, verification references, authorization records, or current environment states. The reliability of these alignment models depends on how requirements are instantiated, how relevant behavior is represented, and how the decision conditions are constructed, evaluated, and updated. The next section examines the resulting challenges in requirement interpretation, multi-requirement alignment, representation, runtime overhead, and intervention.

5 Challenges and Future Directions

Alignment-Based AD addresses a limitation of Boundary-Modeling AD by evaluating current behavior against the goals and constraints specified by applicable requirement instances under the execution context. As discussed in Section 3, LLM-generated outputs and action choices can vary for the same request. Different behavior patterns may conform to expected behavior without resembling historical normal patterns, while similar output formats or execution structures can conceal anomalous behavior. Decision conditions based solely on similarity between current behavior patterns and historical normal patterns may not reliably distinguish normal from anomalous behavior under the current execution context. The alignment model in Section 4 instead establishes decision conditions for evaluating goal achievement and constraint satisfaction. However,

constructing a reliable alignment model still requires interpreting requirements, handling their dependencies and conflicts, and preserving features that indicate requirement violations. Model construction, evaluation, and updates also incur resource overhead, while interventions triggered by anomaly decisions can interfere with reasoning and self-correction. We therefore organize the following challenges from requirement formulation and alignment-model construction to resource use and intervention during execution.

Uncertainty in Requirements. Alignment-Based AD expresses expected behavior through requirements and uses their goals and constraints to evaluate behavior patterns. Requirements can themselves be ambiguous, include implicit goals, or depend on the execution context. Different interpretations of the same user intent can produce different requirement instances and lead to different anomaly decisions for the same behavior pattern. Uncertainty in the evaluation criteria can therefore compound uncertainty in LLM behavior. Future work should study how execution context and user clarification can resolve ambiguous goals and constraints during requirement instantiation. Evaluation should examine whether the resulting requirement instances preserve the intended goals and constraints and how unresolved ambiguity affects anomaly decisions.

Multi-Requirement Alignment. Most methods reviewed in Section 4 construct and evaluate an alignment model for one requirement dimension at a time. However, SLO, user intent, and safety requirements can impose dependent or conflicting goals and constraints on the same behavior pattern. For example, additional tool calls may help fulfill user intent while causing execution cost to exceed an SLO budget. Separate anomaly decisions do not capture how meeting one requirement affects the satisfaction of another. Future work should jointly model correspondences between requirements and behavior patterns together with dependencies and conflicts among the applicable goals and constraints. Such models should evaluate how satisfying one requirement affects the others and retain separate results for each applicable goal and constraint. Each result should identify its supporting evidence and the requirement-instance and alignment-model versions used for evaluation. A mandatory constraint violation should be reported as an anomaly even when other goals are achieved.

Distortion in Alignment. An alignment model establishes correspondences between requirements and behavior patterns, but the mapping process may amplify similarities between their representations while weakening features relevant to anomaly detection. For example, representing a tool call only by its task-related purpose may obscure a parameter that violates a constraint. Representational closeness can then be mistaken for conformity to expected behavior, allowing an anomalous pattern to pass the check. Future work should investigate mapping methods that retain features indicating requirement violations and evaluate whether these violations remain detectable after alignment, reducing false negatives introduced by the mapping.

Overhead in Alignment. Alignment-Based AD uses current runtime data and execution context to construct, evaluate, and update the alignment model. Evidence retrieval and execution-state updates are already part of several reviewed methods [58, 150, 202]. More comprehensive context acquisition and more frequent model updates can improve detection effectiveness, but they also increase retrieval, inference, and verification costs. Reducing these costs may omit information relevant to a decision or delay adaptation to execution changes. Future work should study incremental updates, reuse of prior results, and selective verification to balance resource overhead against detection effectiveness. Evaluation should consider anomaly decisions and detection delay alongside the resources required for model construction, evaluation, and updates.

Interference from Intervention. LLM-based systems may recover from an early anomaly through subsequent reasoning and self-correction. A behavior pattern that deviates from expected behavior at an intermediate step does not necessarily imply final task failure. If a detector blocks an action, triggers a retry, or revises a plan too early, the intervention can interrupt the original reasoning

trajectory and reduce opportunities for exploration or recovery. Future work should consider subsequent execution and task outcomes when studying the timing of intervention. The objective is to control anomaly risks while preserving the system's ability to explore alternative paths and correct recoverable errors.

6 Conclusion

This survey reviewed anomaly detection methods for LLM-based systems and distinguished Boundary-Modeling AD from Alignment-Based AD according to how they establish decision conditions. Both paradigms evaluate whether behavior patterns conform to expected behavior [17]. Boundary-Modeling AD pre-models a boundary from statistical, structural, or feature regularities in historical runtime data and checks subsequent behavior patterns against it. The same user intent can be fulfilled through different retrieved evidence, reasoning paths, and tool calls. Conversely, a response format or tool-call structure resembling a historical normal execution can fail to fulfill the current request. These variations weaken the correspondence between historical normal patterns and expected behavior, allowing valid and anomalous behavior patterns to overlap in a chaotic boundary. Alignment-Based AD addresses this limitation by evaluating current behavior against the goals and constraints expressed through SLO, user intent, and safety requirements. Its alignment model connects applicable requirement instances with current behavior patterns under the execution context and specifies decision conditions for evaluating goal achievement and constraint satisfaction. The model is updated as requirements, runtime data, or context change. We developed taxonomies for both paradigms and a five-stage framework for alignment-model construction, evaluation, and evolution. Future work should reduce uncertainty in requirement instantiation, jointly evaluate interacting requirements, and preserve features indicating requirement violations during alignment. These advances should balance detection effectiveness with resource overhead and support intervention that controls anomaly risks while preserving exploration and recovery.

References

[1] Basel Abdeen, S. M. Tahmid Siddiqui, Meah Tahmeed Ahmed, Anoop Singhal, Latifur Khan, Punya Parag Modi, and Ehab Al-Shaer. 2025. Hallucination Detection in Large Language Models Using Diversion Decoding. In *Data and Applications Security and Privacy XXXIX*. Springer Nature Switzerland, 116–133. doi:10.1007/978-3-031-96590-6_7

[2] Mahmoud Abdelfattah, Hamid Nasiri, and Peter Garraghan. 2026. kNNGuard: Turning LLM Hidden Activations into a Training-Free Configurable Guardrail. arXiv:2607.02072 [cs.LG] https://arxiv.org/abs/2607.02072

[3] Laksh Advani. 2026. Trajectory Guard – A Lightweight, Sequence-Aware Model for Real-Time Anomaly Detection in Agentic AI. doi:10.48550/ARXIV.2601.00516

[4] Laksh Advani. 2026. When Small Models Are Right for Wrong Reasons: Process Verification for Trustworthy Agents. doi:10.48550/ARXIV.2601.00513

[5] Ander Alvarez, Santhiya Rajan, Samuel Mugel, and Román Orús. 2026. ProvenanceGuard: Source-Aware Factuality Verification for MCP-Based LLM Agents. arXiv:2606.18037 [cs.AI] https://arxiv.org/abs/2606.18037

[6] Tyler Alvarez and Ali Baheri. 2026. Where Does Reasoning Break? Step-Level Hallucination Detection via Hidden-State Transport Geometry. *CoRR* abs/2605.13772 (2026). arXiv:2605.13772 doi:10.48550/ARXIV.2605.13772

[7] Chenyang An, Sam Bayless, Stefano Buliani, Darion Cassel, Byron Cook, Duncan Clough, Rémi Delmas, Nafi Diallo, Ferhat Erata, Nick Feng, Dimitra Giannakopoulou, Aman Goel, Aditya Gokhale, Joe Hendrix, Victor Heorhiadi, Marc Hudak, Dejan Jovanović, Andrew M. Kent, Benjamin Kiesl-Reiter, Jeffrey J. Kuna, Nadia Labai, Joseph Lilien, Divya Raghunathan, Zvonimir Rakamarić, Niloofar Razavi, Michael Tautschnig, Ali Torkamani, Nathaniel Weir, Michael W. Whalen, and Jianan Yao. 2025. A Neurosymbolic Approach to Natural Language Formalization and Verification. doi:10.48550/ARXIV.2511.09008

[8] Moises Andrade, Joonhyuk Cha, Brandon Ho, Vriksha Srihari, Karmesh Yadav, and Zsolt Kira. 2025. Let's Think in Two Steps: Mitigating Agreement Bias in MLLMs with Self-Grounded Verification. doi:10.48550/ARXIV.2507.11662

[9] Raed Awill, Wajahat Ali Khan, Maqbool Hussain, Ben Anderson, and S. M Ahsan Kazmi. 2026. ATLASky-AI: An autonomous framework for physics-based trustworthy verification of LLM-generated spatiotemporal knowledge. *Expert Systems with Applications* 318 (July 2026), 131801. doi:10.1016/j.eswa.2026.131801

[10] Abid Aziz and Hafsa Binte Kibria. 2026. RecurGuard: Runtime Monitoring for Reasoning-Token Consumption Attacks. arXiv:2606.07968 [cs.CR] https://arxiv.org/abs/2606.07968

[11] Zepeng Bao, Shen Zhou, Qiankun Pi, Jianhao Chen, Mayi Xu, Ming Zhong, Yuanyuan Zhu, and Tieyun Qian. 2025. Can a Small Model Learn to Look Before It Leaps? Dynamic Learning and Proactive Correction for Hallucination Detection. doi:10.48550/ARXIV.2511.05854

[12] Guy Bar-Shalom, Fabrizio Frasca, Yaniv Galron, Yftah Ziser, and Haggai Maron. 2025. Beyond Token Probes: Hallucination Detection via Activation Tensors with ACT-ViT. doi:10.48550/ARXIV.2510.00296

[13] Abhinaba Basu. 2026. Tool Receipts, Not Zero-Knowledge Proofs: Practical Hallucination Detection for AI Agents. doi:10.48550/ARXIV.2603.10060

[14] Benedikt Bollig. 2026. Causal Past Logic for Runtime Verification of Distributed LLM Agent Workflows. arXiv:2605.20923 [cs.LO] https://arxiv.org/abs/2605.20923

[15] Naci Cankaya. 2026. Bit-Exact AI Inference Verification Without Performance Tradeoffs. arXiv:2606.00279 [cs.CR] https://arxiv.org/abs/2606.00279

[16] Diogo Carvalho, Senka Krivic, Timothy Tang, and Salman Ahmad. 2026. User-State Verification in Conversational Commerce: Detecting Journey Hallucinations via Trace Invariants. In *Proceedings of the 34th ACM Conference on User Modeling, Adaptation and Personalization (UMAP '26)*. ACM, 552–555. doi:10.1145/3774935.3812704

[17] Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. *ACM computing surveys (CSUR)* 41, 3 (2009), 1–58.

[18] Ramaswamy Chandramouli and Zack Butcher. 2023. *A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments.* Technical Report NIST SP 800-207A. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207A

[19] Shenxu Chang, Junchi Yu, Weixing Wang, Yongqiang Chen, Jialin Yu, Philip Torr, and Jindong Gu. 2025. TraceDet: Hallucination Detection from the Decoding Trace of Diffusion Large Language Models. doi:10.48550/ARXIV.2510.01274

[20] Bodhisatwa Chatterjee, Drew Zagieboylo, Sana Damani, Siva Hari, and Christos Kozyrakis. 2025. ProofWright: Towards Agentic Formal Verification of CUDA. doi:10.48550/ARXIV.2511.12294

[21] Chao Chen, Kai Liu, Ze Chen, Yi Gu, Yue Wu, Mingyuan Tao, Zhihang Fu, and Jieping Ye. 2024. INSIDE: LLMs' Internal States Retain the Power of Hallucination Detection. doi:10.48550/ARXIV.2402.03744

[22] Feiyi Chen, Zhen Qin, Yingying Zhang, Shuiguang Deng, Yi Xiao, Guansong Pang, and Qingsong Wen. 2024. LARA: A Light and Anti-overfitting Retraining Approach for Unsupervised Time Series Anomaly Detection. In *Proceedings of the ACM Web Conference 2024*. Association for Computing Machinery, 4138–4149. doi:10.1145/3589334.3645472

[23] Kedi Chen, Qin Chen, Jie Zhou, Xinqi Tao, Bowen Ding, Jingwen Xie, Mingchen Xie, Peilong Li, and Zheng Feng. 2025. Enhancing Uncertainty Modeling with Semantic Graph for Hallucination Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 22 (April 2025), 23586–23594. doi:10.1609/aaai.v39i22.34528

[24] Xiang Chen, Chenxi Wang, Yida Xue, Ningyu Zhang, Xiaoyan Yang, Qiang Li, Yue Shen, Lei Liang, Jinjie Gu, and Huajun Chen. 2024. Unified Hallucination Detection for Multimodal Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 3235–3252. doi:10.18653/v1/2024.acl-long.178

[25] Yuyan Chen, Qiang Fu, Yichen Yuan, Zhihao Wen, Ge Fan, Dayiheng Liu, Dongmei Zhang, Zhixu Li, and Yanghua Xiao. 2023. Hallucination Detection: Robustly Discerning Reliable Answers in Large Language Models. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 245–255. doi:10.1145/3583780.3614905

[26] Zhaorun Chen, Francesco Pinto, Minzhou Pan, and Bo Li. 2024. SafeWatch: An Efficient Safety-Policy Following Video Guardrail Model with Transparent Explanations. doi:10.48550/ARXIV.2412.06878

[27] Haoran Cheng, Yunhao Yao, Jinke Song, Zhiqiang Wang, and Lan Zhang. 2026. IntentGuard: Securing MCP-Enabled LLM Agents via Post-Decision Semantic Plan Verification. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. International Foundation for Autonomous Agents and Multiagent Systems. doi:10.65109/pjgh6650

[28] Xiaoxia Cheng, Zeqi Tan, Zhe Zheng, and Weiming Lu. 2025. G2LDetect: A Global-to-Local Approach for Hallucination Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 1 (April 2025), 102–109. doi:10.1609/aaai.v39i1.31985

[29] Alla Chepurova, Yuri Kuratov, Aydar Bulatov, and Mikhail Burtsev. 2024. Prompt Me One More Time: A Two-Step Knowledge Extraction Pipeline with Ontology-Based Verification. In *Proceedings of TextGraphs-17: Graph-based Methods for Natural Language Processing*, Dmitry Ustalov, Yanjun Gao, Alexander Panchenko, Elena Tutubalina, Irina Nikishina, Arti Ramesh, Andrey Sakhovskiy, Ricardo Usbeck, Gerald Penn, and Marco Valentino (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 61–77. https://aclanthology.org/2024.textgraphs-1.5/

[30] Haechan Chong, Jongwon Lee, and Hyemin Ahn. 2026. Robust Task Planning via Failure Detection Using Scene Graph From Multi-View Images. *IEEE Robotics and Automation Letters* 11, 2 (Feb. 2026), 1986–1993. doi:10.1109/lra.

2025.3645659

[31] Zan-Kai Chong, Hiroyuki Ohsaki, and Bryan Ng Cheng Kuan. 2025. Tractable Asymmetric Verification for Large Language Models via Deterministic Replicability. In *2025 IEEE 23rd Student Conference on Research and Development (SCOReD)*. IEEE, 1–6. doi:10.1109/scored68498.2025.11399166

[32] Kexin Chu, Yang Zhou, and Wei Zhang. 2026. MarginGate: Sparse Margin-Triggered Verification for Batch-Invariant LLM Inference. arXiv:2605.30218 [cs.LG] https://arxiv.org/abs/2605.30218

[33] Kamil Ciosek, Nicolò Felicioni, and Sina Ghiassian. 2025. Hallucination Detection on a Budget: Efficient Bayesian Estimation of Semantic Entropy. doi:10.48550/ARXIV.2504.03579

[34] Cristina Cornelio and Mohammed Diab. 2024. Recover: A Neuro-Symbolic Framework for Failure Detection and Recovery. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 12435–12442. doi:10.1109/iros58592.2024.10801853

[35] Denny B. Czejdo, Wiktor B. Daszczuk, and Waldemar Grabski. 2026. Verification of Synchronization of LLM Agents in the Integrated Model of Distributed Systems Using Petri Net. In *Dependable Computer Systems: Methods, Models, and Applications*. Springer Nature Switzerland, 68–79. doi:10.1007/978-3-032-27927-9_7

[36] Sharanya Dasgupta, Sujoy Nath, Arkaprabha Basu, Pourya Shamsolmoali, and Swagatam Das. 2025. HalluShift: Measuring Distribution Shifts towards Hallucination Detection in LLMs. In *2025 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8. doi:10.1109/ijcnn64981.2025.11228484

[37] Yongxin Deng, Zhen Fang, Yixuan Li, and Ling Chen. 2026. Beyond In-Domain Detection: SpikeScore for Cross-Domain Hallucination Detection. *CoRR* abs/2601.19245 (2026). arXiv:2601.19245 doi:10.48550/ARXIV.2601.19245

[38] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 3563–3578. doi:10.18653/v1/2024.findings-acl.212

[39] Yue Ding, Xiaofang Zhu, Tianze Xia, Junfei Wu, Xinlong Chen, Qiang Liu, and Liang Wang. 2025. D^2HScore: Reasoning-Aware Hallucination Detection via Semantic Breadth and Depth Analysis in LLMs. doi:10.48550/ARXIV.2509.11569

[40] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 1285–1298. doi:10.1145/3133956.3134015

[41] Xuefeng Du, Chaowei Xiao, and Yixuan Li. 2024. HaloScope: Harnessing Unlabeled LLM Generations for Hallucination Detection. *CoRR* abs/2409.17504 (2024). arXiv:2409.17504 doi:10.48550/ARXIV.2409.17504

[42] Laïla Elkoussy and Julien Perez. 2026. AgentLTL: A Trace-Verification Framework for Measuring, Enforcing, and Training Procedural Compliance in Tool-Using LLM Agents. arXiv:2607.02599 [cs.SE] https://arxiv.org/abs/2607.02599

[43] Wangxuan Fan, Xiaoyu Nie, and Zhongxiang Dai. 2026. Harness-MU: A Safe, Governed, and Effective Harness for Multi-User LLM Agents. arXiv:2606.21856 [cs.CR] https://arxiv.org/abs/2606.21856

[44] Jiwei Fang, Bin Zhang, Changwei Wang, Jin Wan, and Zhiwei Xu. 2026. Graph of Verification: Structured Verification of LLM Reasoning with Directed Acyclic Graphs. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 36 (March 2026), 30665–30672. doi:10.1609/aaai.v40i36.40322

[45] Xinyue Fang, Zhen Huang, Zhiliang Tian, Minghui Fang, Ziyi Pan, Quntian Fang, Zhihua Wen, Hengyue Pan, and Dongsheng Li. 2025. Zero-resource Hallucination Detection for Text Generation via Graph-based Contextual Knowledge Triples Modeling. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 22 (April 2025), 23868–23877. doi:10.1609/aaai.v39i22.34559

[46] Hao Fei, Meng Luo, Jundong Xu, Shengqiong Wu, Wei Ji, Mong-Li Lee, and Wynne Hsu. 2024. Fine-grained Structural Hallucination Detection for Unified Visual Comprehension and Generation in Multimodal LLM. In *Proceedings of the 1st ACM Multimedia Workshop on Multi-modal Misinformation Governance in the Era of Foundation Models (MM '24)*. ACM, 13–22. doi:10.1145/3689090.3689388

[47] Yijun Feng. 2025. Counterfactual Probing for Hallucination Detection and Mitigation in Large Language Models. doi:10.48550/ARXIV.2508.01862

[48] Fabrizio Frasca, Guy Bar-Shalom, Yftah Ziser, and Haggai Maron. 2025. Neural Message-Passing on Attention Graphs for Hallucination Detection. doi:10.48550/ARXIV.2509.24770

[49] Kazuaki Furumai, Shuichiro Haruta, Kazunori Matsumoto, and Daisuke Kamisaka. 2026. CORTEX: Token-Level Hallucination Detection in RAG via Comparative Internal Representations. arXiv:2606.31033 [cs.CL] https://arxiv.org/abs/2606.31033

[50] Boris Galitsky and Alexander Rybalov. 2026. Neuro-Symbolic Verification for Preventing LLM Hallucinations in Process Control. *Processes* 14, 2 (Jan. 2026), 322. doi:10.3390/pr14020322

[51] Carlos Garcia-Fernandez, Luis Felipe, Monique Shotande, Muntasir Zitu, Aakash Tripathi, Ghulam Rasool, Issam El Naqa, Vivek Rudrapatna, and Gilmer Valdes. 2025. Trustworthy AI for Medicine: Continuous Hallucination Detection and Elimination with CHECK. doi:10.48550/ARXIV.2506.11129

[52] Guangyu Gong and Zizhuang Deng. 2026. PlanGuard: Defending Agents against Indirect Prompt Injection via Planning-based Consistency Verification. doi:10.48550/ARXIV.2604.10134

[53] Ishrith Gowda. 2026. MEMSAD: Gradient-Coupled Anomaly Detection for Memory Poisoning in Retrieval-Augmented Agents. arXiv:2605.03482 [cs.CR] https://arxiv.org/abs/2605.03482

[54] Pinxue Guo, Chongruo Wu, Xinyu Zhou, Lingyi Hong, Zhaoyu Chen, Jinglun Li, Kaixun Jiang, Sen-Ching Samson Cheung, Wei Zhang, and Wenqiang Zhang. 2026. Seeing Is Believing: Rich-Context Hallucination Detection for MLLMs via Backward Visual Grounding. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 37 (March 2026), 30871–30879. doi:10.1609/aaai.v40i37.40345

[55] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. 2025. Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. In *Proceedings of the First Workshop on LLM Security (LLMSEC)*. Association for Computational Linguistics, Vienna, Austria, 101–114. https://aclanthology.org/2025.llmsec-1.8/

[56] Fouzi Harrag, Hemassa Mecheddal, Narimene Ayat, Mohamed Deriche, and Khaled Shaalan. 2026. Governing AI Responsibly: Designing Guardrails for Large Language Models to Ensure Safety and Accuracy. In *2026 IEEE 23rd International Multi-Conference on Systems, Signals & Devices (SSD)*. IEEE, 275–283. doi:10.1109/ssd69655.2026.11558974

[57] Shuo He, Jiawei Jiang, HongBao Liu, and Tao Tang. 2025. Program Verification to Defend Chain-of-Thought Attacks for LLM Services. In *2025 IEEE International Conference on Web Services (ICWS)*. IEEE, 01–07. doi:10.1109/icws67624.2025.00031

[58] Xu He, Di Wu, Yan Zhai, and Kun Sun. 2025. SentinelAgent: Graph-based Anomaly Detection in Multi-Agent Systems. doi:10.48550/ARXIV.2505.24201

[59] Janek Herrlein, Chia-Chien Hung, and Goran Glavaš. 2024. ANHALTEN: Cross-Lingual Transfer for German Token-Level Reference-Free Hallucination Detection. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*. Association for Computational Linguistics, 186–194. doi:10.18653/v1/2024.acl-srw.18

[60] Yining Hong, Yining She, Eunsuk Kang, Christopher S. Timperley, and Christian Kästner. 2026. Don't Make Models Guess Security and Safety: Symbolic Guardrails for Domain-Specific AI Agents. doi:10.48550/ARXIV.2604.15579

[61] Bairu Hou, Yang Zhang, Jacob Andreas, and Shiyu Chang. 2025. A Probabilistic Framework for LLM Hallucination Detection via Belief Tree Propagation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 3076–3099. doi:10.18653/v1/2025.naacl-long.158

[62] Junjie Hu, Gang Tu, ShengYu Cheng, Jinxin Li, Jinting Wang, Rui Chen, Zhilong Zhou, and Dongbo Shan. 2025. HARP: Hallucination Detection via Reasoning Subspace Projection. doi:10.48550/ARXIV.2509.11536

[63] Wenjing Hu, Weiwei Qi, Yanlu Li, and Xinzhe Huang. 2025. CVshield: Interpretable Black-Box Adversarial Defense for LLMs via CoT Guided Semantic Verification. In *2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 656–663. doi:10.1109/trustcom66490.2025.00079

[64] Xiangkun Hu, Dongyu Ru, Lin Qiu, Qipeng Guo, Tianhang Zhang, Yang Xu, Yun Luo, Pengfei Liu, Yue Zhang, and Zheng Zhang. 2024. Knowledge-Centric Hallucination Detection. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 6953–6975. doi:10.18653/v1/2024.emnlp-main.395

[65] Xiaomeng Hu, Yiming Zhang, Ru Peng, Haozhe Zhang, Chenwei Wu, Gang Chen, and Junbo Zhao. 2024. Embedding and Gradient Say Wrong: A White-Box Method for Hallucination Detection. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1950–1959. doi:10.18653/v1/2024.emnlp-main.116

[66] Tao Huang, Zhekun Liu, Rui Wang, Yang Zhang, and Liping Jing. 2025. Visual hallucination detection in large vision-language models via evidential conflict. *International Journal of Approximate Reasoning* 186 (Nov. 2025), 109507. doi:10.1016/j.ijar.2025.109507

[67] Yiming Huang, Junyan Zhang, Zihao Wang, Biquan Bie, Yunzhong Qiu, Xuming Hu, Yi R. Fung, and Xinlei He. 2025. RePPL: Recalibrating Perplexity by Uncertainty in Semantic Propagation and Language Generation for Explainable QA Hallucination Detection. doi:10.48550/ARXIV.2505.15386

[68] Khizar Hussain and Murat Kantarcioglu. 2026. PARALLAX: Separating Genuine Hallucination Detection from Benchmark Construction Artifacts. arXiv:2605.17028 [cs.CL] https://arxiv.org/abs/2605.17028

[69] ISO/IEC. 2023. *ISO/IEC 25010:2023 Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model*. Technical Report ISO/IEC 25010:2023. International Organization for Standardization. https://www.iso.org/standard/78176.html

[70] David O. Johnston, Arkajyoti Chakraborty, and Nora Belrose. 2025. Mechanistic Anomaly Detection for "Quirky" Language Models. doi:10.48550/ARXIV.2504.08812

[71] Nafiseh Kahani, Masoud Barati, and Diana Addae. 2026. Runtime Compliance Verification for AI Agents. arXiv:2606.19242 [cs.SE] https://arxiv.org/abs/2606.19242

[72] Federico A. Kamelhar. 2026. GSAR: Typed Grounding for Hallucination Detection and Recovery in Multi-Agent LLMs. doi:10.48550/ARXIV.2604.23366

[73] Mintong Kang and Bo Li. 2024. R^2-Guard: Robust Reasoning Enabled LLM Guardrail via Knowledge-Enhanced Logical Reasoning. arXiv:2407.05557 [cs.AI] https://arxiv.org/abs/2407.05557

[74] Adam Karvonen, Daniel Reuter, Roy Rinberg, Luke Marks, Adrià Garriga-Alonso, and Keri Warr. 2025. DiFR: Inference Verification Despite Nondeterminism. doi:10.48550/ARXIV.2511.20621

[75] Hugo Lyons Keenan, Christopher Leckie, and Sarah Erfani. 2026. Mechanistic Anomaly Detection via Functional Attribution. doi:10.48550/ARXIV.2604.18970

[76] Nikita Kezins, Urbas Ekka, Pascal Berrang, and Luca Arnaboldi. 2026. Beyond Red-Teaming: Formal Guarantees of LLM Guardrail Classifiers. doi:10.48550/ARXIV.2605.10901

[77] Gauri Kholkar and Ratinder Ahuja. 2025. Policy-as-Prompt: Turning AI Governance Rules into Guardrails for AI Agents. doi:10.48550/ARXIV.2509.23994

[78] Ragini Kihlman and Maria Fasli. 2025. Hallucination Detection and Reclassification Using Graph Co-Training. In *2025 IEEE International Conference on Big Data (BigData)*. IEEE, 4623–4632. doi:10.1109/bigdata66926.2025.11402538

[79] Hyeong-Geun Kim, Tea-Sung Jun, and Taeseon Lee. 2026. Atomic Contrastive Verification: Fine-Grained Fact-Checking via Claim Decomposition and Knowledge Graph-Grounded Contrastive Reasoning. *Mathematics* 14, 10 (May 2026), 1769. doi:10.3390/math14101769

[80] Minbeom Kim, Mihir Parmar, Phillip Wallis, Lesly Miculicich, Kyomin Jung, Krishnamurthy Dj Dvijotham, Long T. Le, and Tomas Pfister. 2026. CausalArmor: Efficient Indirect Prompt Injection Guardrails via Causal Attribution. doi:10.48550/ARXIV.2602.07918

[81] Shayan Kiyani, Sima Noorani, George Pappas, and Hamed Hassani. 2026. When to Trust the Cheap Check: Weak and Strong Verification for Reasoning. doi:10.48550/ARXIV.2602.17633

[82] Linggang Kong, Yunlong Zhang, Xiaofeng Zhong, Haoran Fu, Yongjie Wang, and Huijun Liu. 2026. HaluGNN: Hallucination detection in large language models using graph neural network. *Expert Systems with Applications* 306 (April 2026), 130857. doi:10.1016/j.eswa.2025.130857

[83] Roham Koohestani. 2025. AgentGuard: Runtime Verification of AI Agents. In *Proceedings of the 2025 IEEE/ACM 40th International Conference on Automated Software Engineering Workshops*. 74–77. doi:10.1109/ASEW67777.2025.00023

[84] Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. 2024. Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs. doi:10.48550/ARXIV.2406.15927

[85] Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. 2026. LLM-as-a-Verifier: A General-Purpose Verification Framework. arXiv:2607.05391 [cs.AI] https://arxiv.org/abs/2607.05391

[86] Qianlong Lan, Anuj Kaul, Shaun Jones, and Stephanie Westrum. 2026. Zero-Trust Runtime Verification for Agentic Payment Protocols: Mitigating Replay and Context-Binding Failures in AP2. doi:10.48550/ARXIV.2602.06345

[87] Arnaud Lawson. 2019. Case Study: Implementing SLOs for a New Service. In *SREcon19 Americas*. USENIX Association. https://www.usenix.org/conference/srecon19americas/presentation/lawson

[88] Alexander W. Lee, Benjamin Han, Shayak Sen, Sam Yeom, Ugur Cetintemel, and Anupam Datta. 2026. Evergreen: Efficient Claim Verification for Semantic Aggregates. doi:10.48550/ARXIV.2604.26180

[89] Jungjae Lee, Dongjae Lee, Chihun Choi, Youngmin Im, Jaeyoung Wi, Kihong Heo, Sangeun Oh, Sunjae Lee, and Insik Shin. 2025. VeriSafe Agent: Safeguarding Mobile GUI Agent via Logic-based Action Verification. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking (ACM MOBICOM '25)*. ACM, 817–831. doi:10.1145/3680207.3765248

[90] Joosung Lee, Cheonbok Park, Hwiyeol Jo, Jeonghoon Kim, Joonsuk Park, and Kang Min Yoo. 2025. Enhancing Hallucination Detection via Future Context. doi:10.48550/ARXIV.2507.20546

[91] Tobias Leemann, Periklis Petridis, Giuseppe Vietri, Dionysis Manousakas, Aaron Roth, and Sergul Aydore. 2024. Auto-GDA: Automatic Domain Adaptation for Efficient Grounding Verification in Retrieval-Augmented Generation. doi:10.48550/ARXIV.2410.03461

[92] Natan Levy, Adiel Ashrov, and Guy Katz. 2025. Statistical Runtime Verification for LLMs via Robustness Estimation. In *Runtime Verification*. Springer Nature Switzerland, 457–476. doi:10.1007/978-3-032-05435-7_25

[93] Ashley Lewis, Andrew Perrault, Eric Fosler-Lussier, and Michael White. 2025. VISTA: Verification In Sequential Turn-based Assessment. doi:10.48550/ARXIV.2510.27052

[94] Hao Li, Xiaogeng Liu, and Chaowei Xiao. 2024. InjecGuard: Benchmarking and Mitigating Over-Defense in Prompt Injection Guardrail Models. *arXiv preprint arXiv:2410.22770* (2024). arXiv:2410.22770 https://arxiv.org/abs/2410.22770v1

[95] Jinxin Li, Gang Tu, ShengYu Cheng, Junjie Hu, Jinting Wang, Rui Chen, Zhilong Zhou, and Dongbo Shan. 2025. LLM Hallucination Detection: A Fast Fourier Transform Method Based on Hidden Layer Temporal Signals. doi:10.48550/ARXIV.2509.13154

[96] Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Yingqiang Ge, Weijie Xu, Xi Fang, Xiang Xu, Tianchen Zhao, Youngeun Kim, Tianyang Wang, Jihun Hamm, and Chandan K. Reddy. 2026. Agent Harness Engineering: A Survey. Manuscript under review.

[97] Ningke Li, Yuekang Li, Yi Liu, Ling Shi, Kailong Wang, and Haoyu Wang. 2024. Drowzee: Metamorphic Testing for Fact-Conflicting Hallucination Detection in Large Language Models. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (Oct. 2024), 1843–1872. doi:10.1145/3689776

[98] Qing Li, Jiahui Geng, Zongxiong Chen, Derui Zhu, Yuxia Wang, Congbo Ma, Chenyang Lyu, and Fakhri Karray. 2025. HD-NDEs: Neural Differential Equations for Hallucination Detection in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6173–6186. doi:10.18653/v1/2025.acl-long.309

[99] Wenyun Li, Zheng Zhang, Dongmei Jiang, and Xiangyuan Lan. 2026. Bolster Hallucination Detection via Prompt-Guided Data Augmentation. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 44 (March 2026), 37618–37626. doi:10.1609/aaai.v40i44.41096

[100] Zhenwen Liang, Ruosen Li, Yujun Zhou, Linfeng Song, Dian Yu, Xinya Du, Haitao Mi, and Dong Yu. 2025. CLUE: Non-parametric Verification from Experience via Hidden-State Clustering. doi:10.48550/ARXIV.2510.01591

[101] Shanshan Lin, Dongsheng Hong, Sibo Ju, Chao Chen, Sihong Xie, and Xiangwen Liao. 2026. Cross Paraphrastic Invariance Learning for Hallucination Detection. *CoRR* abs/2606.08157 (2026). arXiv:2606.08157 doi:10.48550/ARXIV.2606.08157

[102] Chengwu Liu, Ye Yuan, Yichun Yin, Yan Xu, Xin Xu, Zaoyu Chen, Yasheng Wang, Lifeng Shang, Qun Liu, and Ming Zhang. 2025. Safe: Enhancing Mathematical Reasoning in Large Language Models via Retrospective Step-aware Formal Verification. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 12171–12186. doi:10.18653/v1/2025.acl-long.594

[103] He Liu, Changtao Miao, Xinjie Yang, Tianle Song, Yin Wu, Junchi Chen, Bintao He, Xinyuan Zhang, Bo Zhang, Shi Yan, Wei Lu, Wei Wang, Danyang Xu, Jiansheng Cai, and Zhe Li. 2026. DT-Guard: Intent-Driven Reasoning-Active Training for Reasoning-Free LLM Safety Guardrail. arXiv:2607.06326 [cs.AI] https://arxiv.org/abs/2607.06326

[104] Jiahao Liu, Bonan Ruan, Xianglin Yang, Zhiwei Lin, Yan Liu, Yang Wang, Tao Wei, and Zhenkai Liang. 2025. TraceAegis: Securing LLM-Based Agents via Hierarchical and Behavioral Anomaly Detection. *arXiv preprint arXiv:2510.11203* (2025). arXiv:2510.11203 https://arxiv.org/abs/2510.11203

[105] Litian Liu, Reza Pourreza, Sunny Panchal, Apratim Bhattacharyya, Yubing Jian, Yao Qin, and Roland Memisevic. 2025. Enhancing Hallucination Detection through Noise Injection. doi:10.48550/ARXIV.2502.03799

[106] Ping Liu, Haowen Xu, Qianyu Ouyang, Rui Jiao, Zhekang Chen, Shenglin Zhang, Jiahai Yang, Linlin Mo, Jice Zeng, Wenman Xue, and Dan Pei. 2020. Unsupervised Detection of Microservice Trace Anomalies through Service-Level Deep Bayesian Networks. In *2020 IEEE 31st International Symposium on Software Reliability Engineering*. 48–58. doi:10.1109/ISSRE5003.2020.00014

[107] Qiang Liu, Xinlong Chen, Yue Ding, Bowen Song, Weiqiang Wang, Shu Wu, and Liang Wang. 2025. Attention-guided Self-reflection for Zero-shot Hallucination Detection in Large Language Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 21016–21032. doi:10.18653/v1/2025.emnlp-main.1063

[108] Yuyang Liu, Jingjing Cai, Jiayi Ren, Peng Zhou, Danyang Zhang, Yin Du, and Shijian Li. 2025. Kunlun Anomaly Troubleshooter: Enabling Kernel-Level Anomaly Detection and Causal Reasoning for Large Model Distributed Inference. doi:10.48550/ARXIV.2511.05978

[109] Yibing Liu, Chong Zhang, Zhongyi Han, Hansong Liu, Yong Wang, Yang Yu, Xiaoyan Wang, and Yilong Yin. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. doi:10.48550/ARXIV.2602.06443

[110] Zihang Liu, Jiawei Guo, Hao Zhang, Hongyang Chen, Jiajun Bu, and Haishuai Wang. 2025. Long-form Hallucination Detection with Self-elicitation. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 4082–4100. doi:10.18653/v1/2025.findings-acl.211

[111] Qinghua Lu, Liming Zhu, Xiwei Xu, Zhenchang Xing, and Jon Whittle. 2023. A Reference Architecture for Designing Foundation Model Based Systems. *arXiv preprint arXiv:2304.11090* (2023). arXiv:2304.11090 https://arxiv.org/abs/2304.11090

[112] Yao Lu, Song Bian, Lequn Chen, Yongjun He, Yulong Hui, Matthew Lentz, Beibin Li, Fei Liu, Jialin Li, Qi Liu, Rui Liu, Xiaoxuan Liu, Lin Ma, Kexin Rong, Jianguo Wang, Yingjun Wu, Yongji Wu, Huanchen Zhang, Minjia Zhang, Qizhen Zhang, Tianyi Zhou, and Danyang Zhuo. 2024. Computing in the Era of Large Generative Models: From Cloud-Native to AI-Native. *arXiv preprint arXiv:2401.12230* (2024). arXiv:2401.12230 https://arxiv.org/abs/2401.12230

[113] Yu Lu, Zhoubing Xiong, Helin Huang, Bing He, and Jinjun Kuang. 2025. Harmonizing Parameterized Verification and Invariant Synthesis for Safety Envelopes in Black-Box Autonomy. *Concurrency and Computation: Practice and Experience* 37, 27-28 (Oct. 2025). doi:10.1002/cpe.70339

[114] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. 2025. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 8104–8139. doi:10.18653/v1/2025.acl-long.399

[115] Jiaju Ma and Maneesh Agrawala. 2025. MoVer: Motion Verification for Motion Graphics Animations. *ACM Transactions on Graphics* 44, 4 (July 2025), 1–17. doi:10.1145/3731209

[116] Minghua Ma, Shenglin Zhang, Junjie Chen, Jim Xu, Haozhe Li, Yongliang Lin, Xiaohui Nie, Bo Zhou, Yong Wang, and Dan Pei. 2021. Jump-Starting Multivariate Time Series Anomaly Detection for Online Service Systems. In *2021 USENIX Annual Technical Conference*. USENIX Association, 413–426. https://www.usenix.org/conference/atc21/presentation/ma

[117] Minghua Ma, Shenglin Zhang, Dan Pei, Xin Huang, and Hongwei Dai. 2018. Robust and Rapid Adaption for Concept Drift in Software System Anomaly Detection. In *2018 IEEE 29th International Symposium on Software Reliability Engineering*. IEEE, 13–24. doi:10.1109/ISSRE.2018.00013

[118] Zhiyuan Ma, Jiayu Liu, Xianzhen Luo, Zhenya Huang, Qingfu Zhu, and Wanxiang Che. 2025. Advancing Tool-Augmented Large Language Models via Meta-Verification and Reflection Learning. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Vol. 2. 2078–2089. doi:10.1145/3711896.3736835

[119] Netta Madvil, Gilad Dym, Alon Mecilati, Edo Dekel, Jonatan Liberman, Rotem Brazilay, Liron Schliesser, Max Svidlo, Shai Nir, Orel Shalom, Yaron Friedman, David Connack, Amos Rimon, Philip Tannor, and Shir Chorev. 2026. Holistic Evaluation and Failure Diagnosis of AI Agents. arXiv:2605.14865 [cs.AI] https://arxiv.org/abs/2605.14865

[120] Alexandre Cristovão Maiorano. 2026. LLM Readiness Harness: Evaluation, Observability, and CI Gates for LLM/RAG Applications. doi:10.48550/ARXIV.2603.27355

[121] Potsawee Manakul, Adian Liusie, and Mark Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 9004–9017. doi:10.18653/v1/2023.emnlp-main.557

[122] Chandrashekhar Medicherla, Chaitanya Kulkarni, Tejas Pravinbhai Patel, Sandeep Shivam, Venkat Nutalapati, Milan Parikh, Madhushree Kumari, and Arun Kumar Elengovan. 2026. ContextCheck: A Self-Consistency Framework for Hallucination Detection in Retrieval-Augmented Generation Systems. In *Proceedings of the International Seminar on Intelligent Business and Edge-Computing Research*. 19–24. doi:10.1109/ISIBER68248.2026.11470068

[123] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*. 4739–4745. doi:10.24963/ijcai.2019/658

[124] Alfredo Metere. 2026. Methods for Formal Verification of Agent Skills: Three Layers Toward a Mechanically Checkable Capability-Containment Proof. doi:10.48550/ARXIV.2605.23951

[125] Rui Min, Tianyu Pang, Chao Du, Minhao Cheng, and Yi R. Fung. 2026. Scalable Token-Level Hallucination Detection in Large Language Models. arXiv:2605.12384 [cs.CL] https://arxiv.org/abs/2605.12384

[126] Amir Hameed Mir. 2025. The Geometry of Truth: Layer-wise Semantic Dynamics for Hallucination Detection in Large Language Models. doi:10.48550/ARXIV.2510.04933

[127] Royce Moon and Lav R. Varshney. 2026. Containment Verification: AI Safety Guarantees Independent of Alignment. doi:10.48550/ARXIV.2605.09045

[128] Charles Moslonka, Hicham Randrianarivo, Arthur Garnier, and Emmanuel Malherbe. 2026. Learned Hallucination Detection in Black-Box LLMs Using Token-Level Entropy Production Rate. In *Advances in Information Retrieval*. Springer Nature Switzerland, 115–130. doi:10.1007/978-3-032-21289-4_8

[129] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based agents via Proactive Step-level Guardrail and Feedback. doi:10.48550/ARXIV.2601.10156

[130] Hadar Mulian, Sergey Zeltyn, Ido Levy, Liane Galanti, Avi Yaeli, and Segev Shlomov. 2026. AgentFixer: From Failure Detection to Fix Recommendations in Agentic Systems. In *Proceedings of the 2026 International Workshop on Agentic Engineering (AGENT '26)*. ACM, 96–103. doi:10.1145/3786167.3788427

[131] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. 2024. RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 10862–10878. doi:10.18653/v1/2024.acl-long.585

[132] Mengjia Niu, Hamed Haddadi, and Guansong Pang. 2025. Robust Hallucination Detection in LLMs via Adaptive Token Selection. doi:10.48550/ARXIV.2504.07863

[133] Valentin Noël. 2026. Spectral Guardrails for Agents in the Wild: Detecting Tool Use Hallucinations via Attention Topology. doi:10.48550/ARXIV.2602.08082

[134] Valentin Noël and Elimane Yassine Seidou. 2026. ReGA: Zero-Overhead Graph Alignment for Structural Hallucination Detection Without Generation. In *Companion Proceedings of the ACM Web Conference 2026*. ACM, 617–625. doi:10.1145/3774905.3794657

[135] Junjun Pan, Yixin Liu, Rui Miao, Kaize Ding, Yu Zheng, Quoc Viet Hung Nguyen, Alan Wee-Chung Liew, and Shirui Pan. 2025. Explainable and Fine-Grained Safeguarding of LLM Multi-Agent Systems via Bi-Level Graph Anomaly Detection. doi:10.48550/ARXIV.2512.18733

[136] Subham Panda. 2026. Citation Discipline in Spec-Driven Development: A Cross-Model Empirical Study of Output Determinism and Automated Hallucination Detection in LLM-Generated Code. arXiv:2606.30689 [cs.SE] https://arxiv.org/abs/2606.30689

[137] Ravi Pandya, Madison Bland, Duy P. Nguyen, Changliu Liu, Jaime Fernández Fisac, and Andrea Bajcsy. 2025. From Refusal to Recovery: A Control-Theoretic Approach to Generative AI Guardrails. doi:10.48550/ARXIV.2510.13727

[138] Seongheon Park, Xuefeng Du, Min-Hsuan Yeh, Haobo Wang, and Yixuan Li. 2025. How to Steer LLM Latents for Hallucination Detection? *CoRR* abs/2503.01917 (2025). arXiv:2503.01917 doi:10.48550/ARXIV.2503.01917

[139] Yu Pei, Yongping Du, and Xingnan Jin. 2025. FoVer: First-Order Logic Verification for Natural Language Reasoning. *Transactions of the Association for Computational Linguistics* 13 (2025), 1340–1359. doi:10.1162/tacl.a.41

[140] Jasmine Qi, Danylo Dantsev, and Muyang Sun. 2026. VERDI: Single-Call Confidence Estimation for Verification-Based LLM Judges via Decomposed Inference. arXiv:2605.11334 [cs.LG] https://arxiv.org/abs/2605.11334

[141] Yanyu Qian, Yue Tan, Yixin Liu, Wang Yu, and Shirui Pan. 2026. DynHD: Hallucination Detection for Diffusion Large Language Models via Denoising Dynamics Deviation Learning. doi:10.48550/ARXIV.2603.16459

[142] Hezhe Qiao, Hanghang Tong, Ee-Peng Lim, Bing Liu, and Guansong Pang. 2026. VerifyMAS: Hypothesis Verification for Failure Attribution in LLM Multi-Agent Systems. arXiv:2605.17467 [cs.CL] https://arxiv.org/abs/2605.17467

[143] Yitong Qiao, Licheng Pan, Yu Mi, Lei Liu, Yue Shen, Fei Sun, and Zhixuan Chu. 2026. Lowest Span Confidence: A Zero-Shot Metric for Efficient and Black-Box Hallucination Detection in LLMs. doi:10.48550/ARXIV.2601.19918

[144] Jingxi Qiu, Zeyu Han, and Cheng Huang. 2026. SURE-RAG: Sufficiency and Uncertainty-Aware Evidence Verification for Selective Retrieval-Augmented Generation. doi:10.48550/ARXIV.2605.03534

[145] Ella Rabinovich, David Boaz, Naama Zwerdling, and Ateret Anaby-Tavor. 2026. Near-Miss: Latent Policy Failure Detection in Agentic Workflows. doi:10.48550/ARXIV.2603.29665

[146] Keshav Ramani, Vali Tawosi, Salwa Alamir, and Daniel Borrajo. 2025. Bridging LLM Planning Agents and Formal Methods: A Case Study in Plan Verification. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. IEEE, 39–42. doi:10.1109/asew67777.2025.00018

[147] Mohamed Rashad, Ahmed Zahran, Abanoub Amin, Amr Abdelaal, and Mohamed Altantawy. 2024. FactAlign: Fact-Level Hallucination Detection and Classification Through Knowledge Graph Alignment. In *Proceedings of the 4th Workshop on Trustworthy Natural Language Processing (TrustNLP 2024)*. Association for Computational Linguistics, 79–84. doi:10.18653/v1/2024.trustnlp-1.8

[148] Zachary Ravichandran, Alexander Robey, Vijay Kumar, George J. Pappas, and Hamed Hassani. 2026. Safety Guardrails for LLM-Enabled Robots. *IEEE Robotics and Automation Letters* 11, 4 (April 2026), 4649–4656. doi:10.1109/lra.2026.3667488

[149] James Rhodes and George Kang. 2026. Proof of Execution: Runtime Verification for Governed AI Agent Actions. doi:10.48550/ARXIV.2607.05397

[150] Fabio Rovai. 2026. CIVeX: Causal Intervention Verification for Language Agents. doi:10.48550/ARXIV.2605.09168

[151] Michael Saldivar and Ben Slivinski. 2026. Theoria: Rewrite-Acceptability Verification over Informal Reasoning States. arXiv:2607.01223 [cs.AI] https://arxiv.org/abs/2607.01223

[152] Mahya Samdaliri and Zhihao Yao. 2025. PACE: Policy Adaptation for Cybersecurity Events Through Formal Verification of LLM-Generated Firewall Policies. In *2025 IEEE International Conference on Big Data (BigData)*. IEEE, 7835–7844. doi:10.1109/bigdata66926.2025.11401967

[153] Albert Sawczyn, Jakub Binkowski, Denis Janiak, Bogdan Gabrys, and Tomasz Jan Kajdanowicz. 2026. FactSelfCheck: Fact-Level Black-Box Hallucination Detection for LLMs. In *Findings of the Association for Computational Linguistics: EACL 2026*. Association for Computational Linguistics, 5603–5621. doi:10.18653/v1/2026.findings-eacl.296

[154] Arsenios Scrivens. 2026. Empirical Validation of the Classification-Verification Dichotomy for AI Safety Gates. doi:10.5281/ZENODO.19237566

[155] Arsenios Scrivens. 2026. Information-Theoretic Limits of Safety Verification for Self-Improving Systems. doi:10.5281/ZENODO.19237451

[156] Wenlei Shi and Xing Jin. 2025. Heimdall: test-time scaling on the generative verification. doi:10.48550/ARXIV.2504.10337

[157] Channdeth Sok, David Luz, and Yacine Haddam. 2025. MetaRAG: Metamorphic Testing for Hallucination Detection in RAG Systems. doi:10.48550/ARXIV.2509.09360

[158] Aivin V. Solatorio. 2025. Proof-Carrying Numbers (PCN): A Protocol for Trustworthy Numeric Answers from LLMs via Claim Verification. doi:10.48550/ARXIV.2509.06902

[159] Alaia Solko-Breslin, Pramod Kaushik Mudrakarta, Mihai Christodorescu, Somesh Jha, and Krishnamurthy Dj Dvijotham. 2026. Efficient and Sound Probabilistic Verification for AI Agents. arXiv:2606.20510 [cs.CR] https://arxiv.org/abs/2606.20510

[160] Yusheng Song, Lirong Qiu, Xi Zhang, and Zhihao Tang. 2025. Hallucination Detection via Internal States and Structured Reasoning Consistency in Large Language Models. doi:10.48550/ARXIV.2510.11529

[161] Ponhvoan Srey, Xiaobao Wu, and Anh Tuan Luu. 2025. Unsupervised Hallucination Detection by Inspecting Reasoning Processes. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 22128–22140. doi:10.18653/v1/2025.emnlp-main.1124

[162] Weihang Su, Changyue Wang, Qingyao Ai, Yiran Hu, Zhijing Wu, Yujia Zhou, and Yiqun Liu. 2024. Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models. In *Findings of the Association for Computational Linguistics ACL 2024*. Association for Computational Linguistics, 14379–14391. doi:10.18653/v1/2024.findings-acl.854

[163] Yiming Su, Kunzhao Xu, Yanjie Gao, Fan Yang, Cheng Li, Mao Yang, and Tianyin Xu. 2026. Neuro-Symbolic Verification on Instruction Following of LLMs. doi:10.48550/ARXIV.2601.17789

[164] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. 2019. Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2828–2837. doi:10.1145/3292500.3330672

[165] Malavika Suresh, Rahaf Aljundi, Ikechukwu Nkisi-Orji, and Nirmalie Wiratunga. 2025. Cross-Layer Attention Probing for Fine-Grained Hallucination Detection. doi:10.48550/ARXIV.2509.09700

[166] Zhewen Tan, Yilun Yao, Huiyan Jin, Wenhan Yu, Guoan Wang, Mengyuan Fan, liang lu, Feng Liu, Xiangzheng Zhang, Duohe Ma, Tong Yang, and Lin Sun. 2026. MemAudit: Post-hoc Auditing of Poisoned Agent Memory via Causal Attribution and Structural Anomaly Detection. arXiv:2605.23723 [cs.AI] https://arxiv.org/abs/2605.23723

[167] SingGuard Team. 2026. SingGuard: A Policy-Adaptive Multimodal LLM Guardrail with Dynamic Reasoning. arXiv:2606.22873 [cs.CV] https://arxiv.org/abs/2606.22873

[168] Joshua Tint. 2025. PROOD: A Simple LLM Out-of-Distribution Guardrail Leveraging Response Semantics. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 23428–23438. doi:10.18653/v1/2025.findings-emnlp.1272

[169] Chaodong Tong, Qi Zhang, Chen Li, Lei Jiang, and Yanbing Liu. 2026. FaithSCAN: Model-Driven Single-Pass Hallucination Detection for Faithful Visual Question Answering. doi:10.48550/ARXIV.2601.00269

[170] Mohd Safwan Uddin and Saba Hajira. 2026. AegisUI: Behavioral Anomaly Detection for Structured User Interface Protocols in AI Agent Systems. doi:10.48550/ARXIV.2603.05031

[171] Axel van Lamsweerde and Emmanuel Letier. 2000. Handling Obstacles in Goal-Oriented Requirements Engineering. *IEEE Transactions on Software Engineering* 26, 10 (2000), 978–1005. doi:10.1109/32.879820

[172] Iñaki Dellibarda Varela, R. Sendra-Arranz, Pablo Romero-Sorozabal, J. M. Valverde-García, Annemarie F. Laudanski, Álvaro Gutiérrez, Eduardo Rocon, and Manuel Cebrian. 2026. POIROT: Interrogating Agents for Failure Detection in Multi-Agent Systems. arXiv:2606.02282 [cs.AI] https://arxiv.org/abs/2606.02282

[173] Artem Vazhentsev, Lyudmila Rvanova, Gleb Kuzmin, Ekaterina Fadeeva, Ivan Lazichny, Alexander Panchenko, Maxim Panov, Mrinmaya Sachan, Preslav Nakov, Timothy Baldwin, and Artem Shelmanov. 2025. Efficient Hallucination Detection for LLMs Using Uncertainty-Aware Attention Heads. doi:10.48550/ARXIV.2505.20045

[174] Yuiga Wada, Kazuki Matsuda, Komei Sugiura, and Graham Neubig. 2025. ZINA: Multimodal Fine-grained Hallucination Detection and Editing. doi:10.48550/ARXIV.2506.13130

[175] Bo Wang, Gang Zhou, Wentao Yu, and Xi Zhang. 2025. DFV-RAG: Dual-Perspective Fast Verification for Retrieval-Augmented Generation. In *2025 3rd International Conference on Artificial Intelligence and Automation Control (AIAC)*. IEEE, 312–316. doi:10.1109/aiac68175.2025.11332372

[176] Changyue Wang, Weihang Su, Qingyao Ai, and Yiqun Liu. 2026. Joint Evaluation of Answer and Reasoning Consistency for Hallucination Detection in Large Reasoning Models. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 39 (March 2026), 33377–33385. doi:10.1609/aaai.v40i39.40624

[177] Haoyu Wang, Christopher M. Poskitt, Jun Sun, and Jiali Wei. 2025. Pro^{2}Guard: Proactive Runtime Enforcement of LLM Agent Safety via Probabilistic Model Checking. doi:10.48550/ARXIV.2508.00500

[178] Minxing Wang, Xiaofei Xie, and Yintong Huo. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563 [cs.SE] https://arxiv.org/abs/2605.26563

[179] Thomas Wang and Haowen Li. 2025. OpenGuardrails: A Configurable, Unified, and Scalable Guardrails Platform for Large Language Models. doi:10.48550/ARXIV.2510.19169

[180] Xiaohua Wang, Yuliang Yan, Longtao Huang, Xiaoqing Zheng, and Xuanjing Huang. 2023. Hallucination Detection for Generative Large Language Models by Bayesian Sequential Estimation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 15361–15371. doi:10.18653/v1/2023.emnlp-main.949

[181] Zexin Wang, Changhua Pei, Yuanhao Liu, Jingjing Li, Yintong Huo, Quan Zhou, Haotian Si, Hang Cui, Zihan Liu, Gaogang Xie, Fei Sun, Dan Pei, and David Lo. 2026. Agent System Operations: Categorization, Challenges, and Future Directions. *arXiv preprint arXiv:2606.01581* (2026). arXiv:2606.01581 https://arxiv.org/abs/2606.01581

[182] Yangyang Wei, Yijie Xu, Zhenyuan Li, Xiangmin Shen, and Shouling Ji. 2026. Beyond Input Guardrails: Reconstructing Cross-Agent Semantic Flows for Execution-Aware Attack Detection. doi:10.48550/ARXIV.2603.04469

[183] Xiaofei Wen, Wenxuan Zhou, Wenjie Jacky Mo, and Muhao Chen. 2025. ThinkGuard: Deliberative Slow Thinking Leads to Cautious Guardrails. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 13698–13713. doi:10.18653/v1/2025.findings-acl.704

[184] Benlong Wu, Weiming Zhang, Kejiang Chen, Han Fang, and Nenghai Yu. 2026. Provably Secure Agent Guardrail. arXiv:2605.29251 [cs.AI] https://arxiv.org/abs/2605.29251

[185] Yuhao Wu, Tung-Ling Li, and Hongliang Liu. 2026. Behavioral Integrity Verification for AI Agent Skills. *CoRR* abs/2605.11770 (2026). arXiv:2605.11770 doi:10.48550/ARXIV.2605.11770

[186] Zijian Wu, Lingkai Kong, Wenwei Zhang, Songyang Gao, Yuzhe Gu, Zhongrui Cai, Tianyou Ma, Yuhong Liu, Zhi Wang, Runyuan Ma, Guangyu Wang, Wei Li, Conghui He, Dahua Lin, and Kai Chen. 2025. OPV: Outcome-based Process Verifier for Efficient Long Chain-of-Thought Verification. doi:10.48550/ARXIV.2512.10756

[187] Melwin Xavier, Vaisakh M A, Melveena Jolly, and Midhun Xavier. 2026. Agentproof: Static Verification of Agent Workflow Graphs. doi:10.48550/ARXIV.2603.20356

[188] Zhenchang Xing, Qing Huang, Yu Cheng, Liming Zhu, Qinghua Lu, and Xiwei Xu. 2023. Prompt Sapper: LLM-Empowered Software Engineering Infrastructure for AI-Native Services. *arXiv preprint arXiv:2306.02230* (2023). arXiv:2306.02230 https://arxiv.org/abs/2306.02230

[189] Yihao Xue, Kristjan Greenewald, Youssef Mroueh, and Baharan Mirzasoleiman. 2025. Verify when Uncertain: Beyond Self-Consistency in Black Box Hallucination Detection. doi:10.48550/ARXIV.2502.15845

[190] Borui Yang, Md Afif Al Mamun, Jie M. Zhang, and Gias Uddin. 2025. Hallucination Detection in Large Language Models with Metamorphic Relations. *Proceedings of the ACM on Software Engineering* 2, FSE (June 2025), 425–445. doi:10.1145/3715735

[191] Bo Yang, Yinfen Xia, Weisong Sun, and Yang Liu. 2025. Hallucination Detection for LLM-based Text-to-SQL Generation via Two-Stage Metamorphic Testing. doi:10.48550/ARXIV.2512.22250

[192] Jiahao Yang, Shuhai Zhang, Hailong Kang, Feng Liu, Qi Chen, and Mingkui Tan. 2026. Zero-source LLM Hallucination Detection with Human-like Criteria Probing. arXiv:2606.12900 [cs.AI] https://arxiv.org/abs/2606.12900

[193] Shiping Yang, Renliang Sun, and Xiaojun Wan. 2023. A New Benchmark and Reverse Validation Method for Passage-level Hallucination Detection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 3898–3908. doi:10.18653/v1/2023.findings-emnlp.256

[194] Mehdi Yekrangi, Houssem Chatbri, Claudia Beatrice Chianella, and Owen O'Neill. 2026. A Guardrail Framework for Sensitive Financial Information Protection: A Taxonomy-Driven Approach. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 47 (March 2026), 40528–40535. doi:10.1609/aaai.v40i47.41498

[195] Xuanyu Yin, Yilin Jiang, Jun Zhou, Kai Chen, Zhengfu Cao, and Xiaolei Dong. 2026. DoubtProbe: Black-Box Jailbreak Defense via Structural Verification and Semantic Auditing. arXiv:2606.16527 [cs.CR] https://arxiv.org/abs/2606.16527

[196] Boxi Yu, Yuzhong Zhang, Liting Lin, Lionel Briand, and Emir Muñoz. 2026. Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG. doi:10.48550/ARXIV.2603.27752

[197] Jiaqi Yu, Xin Wang, Yixu Wang, Jie Li, Yan Teng, Xingjun Ma, and Yingchun Wang. 2026. SentGuard: Sentence-Level Streaming Guardrails for Large Language Models. arXiv:2606.02041 [cs.CL] https://arxiv.org/abs/2606.02041

[198] Zhuowen Yuan, Zhaorun Chen, Zhen Xiang, Nathaniel D. Bastian, Seyyed Hadi Hashemi, Chaowei Xiao, Wenbo Guo, and Bo Li. 2026. ShieldNet: Network-Level Guardrails against Emerging Supply-Chain Injections in Agentic Systems. doi:10.48550/ARXIV.2604.04426

[199] Bo Zhang, Cong Gao, Bingxu Han, Minghao Hu, Zhunchen Luo, Jun Zhang, Wen Yao, Xiaoying Bai, Guotong Geng, and Zhong Wang. 2026. MathHalu: A Benchmark for Mathematical Reasoning Process Hallucination Detection in Large Reasoning Models. In *2026 IEEE International Conference on Acoustics, Speech and Signal Processing*. doi:10.1109/ICASSP55912.2026.11460764

[200] Dawen Zhang, Xiwei Xu, Chen Wang, Zhenchang Xing, and Robert Mao. 2024. A Layered Architecture for Developing and Enhancing Capabilities in Large Language Model-Based Software Systems. *arXiv preprint arXiv:2411.12357* (2024). arXiv:2411.12357 https://arxiv.org/abs/2411.12357

[201] Fujie Zhang, Peiqi Yu, Biao Yi, Baolei Zhang, Tong Li, and Zheli Liu. 2025. Prompt-Guided Internal States for Hallucination Detection of Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 21806–21818. doi:10.18653/v1/2025.acl-long.1058

[202] Hang Zhang, Ruheng Wang, Yuelyu Ji, Min Gu Kwak, Xizhi Wu, Chenyu Li, Li Zhang, Wenqi Shi, Yifan Peng, and Yanshan Wang. 2026. Scaling Medical Reasoning Verification via Tool-Integrated Reinforcement Learning. *CoRR* abs/2601.20221 (2026). arXiv:2601.20221 doi:10.48550/ARXIV.2601.20221

[203] Jiaxin Zhang, Zhuohang Li, Kamalika Das, Bradley Malin, and Sricharan Kumar. 2023. SAC3: Reliable Hallucination Detection in Black-Box Language Models via Semantic-aware Cross-check Consistency. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 15445–15458. doi:10.18653/v1/2023.findings-emnlp.1032

[204] Ruwen Zhang, Jinglu Chen, Mingjie Dai, Xinyi Jiang, Yuxin Hu, Bo Liu, and Jiuxin Cao. 2024. METER: Multimodal Hallucination Detection with Mixture of Experts via Tools Ensembling and Reasoning. In *Natural Language Processing and Chinese Computing*. Springer Nature Singapore, 274–286. doi:10.1007/978-981-97-9443-0_24

[205] Shunhong Zhang, Gang Lu, and Zichen Ma. 2026. Low-Cost Verification for Multimodal Large Language Models. In *2026 International Conference on Communication Networks and Machine Learning (CNML)*. IEEE, 1158–1161. doi:10.1109/cnml68938.2026.11453286

[206] Yedi Zhang, Sun Yi Emma, Annabelle Lee Jia En, and Jin Song Dong. 2025. RvLLM: LLM Runtime Verification with Domain Knowledge. doi:10.48550/ARXIV.2505.18585

[207] Yulong Zhang, Li Wang, Wei Du, Peilin Li, Yuqin Dai Zhiyuan Zhao, Lingyong Fang, Ziniu Liu, Ru Zhang, Huijia Zhu, and Gongshen Liu. 2025. NCV: A Node-Wise Consistency Verification Approach for Low-Cost Structured Error Localization in LLM Reasoning. doi:10.48550/ARXIV.2510.02816

[208] Yukun Zhang, Yufei Zeng, Yifan Zhu, Meina Song, Wei Wang, and Zhonghong Ou. 2025. TGRV: Triple Graph-Based Retrieval and Verification for Hallucination Detection in Large Language Models. In *2025 International Conference on Data Science and Intelligent Systems (DSIS)*. IEEE, 1–9. doi:10.1109/dsis67228.2025.11390158

[209] Zhenliang Zhang, Xinyu Hu, Huixuan Zhang, Junzhe Zhang, and Xiaojun Wan. 2025. ICR Probe: Tracking Hidden State Dynamics for Reliable Hallucination Detection in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 17986–18002. doi:10.18653/v1/2025.acl-long.880

[210] Zishuai Zhang, Hainan Zhang, and Zhiming Zheng. 2026. AURORA: Asymmetry and Update-Induced Rotation for Robust Hallucination Detection in Large Language Models. arXiv:2606.29545 [cs.CL] https://arxiv.org/abs/2606.29545

[211] Runze Zhao, Fuqing Zhu, Jizhong Han, and Songlin Hu. 2025. Visual Perception Uncertainty Learning for Hallucination Detection in Large Vision-Language Models. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*. ACM, 3595–3604. doi:10.1145/3746027.3755126

[212] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Tao Ma, Dan Williams, Wei Wang, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. arXiv:2606.25189 [cs.OS] https://arxiv.org/abs/2606.25189

[213] Xingfu Zhou and Pengfei Wang. 2025. Reasoning-Style Poisoning of LLM Agents via Stealthy Style Transfer: Process-Level Attacks and Runtime Monitoring in RSV Space. doi:10.48550/ARXIV.2512.14448

[214] Ziling Zhou. 2026. Governing Dynamic Capabilities: Cryptographic Binding and Reproducibility Verification for AI Agent Tool Use. doi:10.48550/ARXIV.2603.14332

[215] Geigh Zollicoffer, Minh Vu, Hongli Zhan, Raymond Li, and Manish Bhattarai. 2026. Sanity Checks for Long-Form Hallucination Detection. doi:10.48550/ARXIV.2605.08346

[216] Ahang Zuo. 2026. VeriFlow: Multi-dimensional Verification of LLM-Generated Low-Code Workflows. In *Business Information Systems*. Springer Nature Switzerland, 155–168. doi:10.1007/978-3-032-26363-6_12

[217] Ádám Kovács, Bowei He, Xue Liu, István Boros, Szilveszter Tóth, and Gábor Recski. 2026. Beyond Document Grounding: Span-Level Hallucination Detection over Code, Tool Output, and Documents. arXiv:2607.00895 [cs.CL] https://arxiv.org/abs/2607.00895

[218] Abdullah Önden. 2026. Hallucination Detection in Large Language Models via Multi-Granular Uncertainty Quantification. *Computer and Decision Making: An International Journal* 3 (March 2026), 805–819. doi:10.59543/comdem.v3i.17665